

GitHub Lab Task Guide

<https://chatgpt.com/share/6ac87bb0-6fa8-83ee-8848-024039140b15>

Date: 2026-10-09 · 91 messages

User

<https://github.com/SETAPESU26/60numberguess>

this is my lab task for today and the topic is the github repo tell me what i need to do with this

Original custom instructions no longer available

ChatGPT

Avi, your Lab 4 is an **individual Vibe Coding assignment**. The goal is to take the Python game assigned to you on GitHub, understand it, fix its broken code using an AI tool like ChatGPT, implement the features listed in its README, and push the completed project to your own GitHub account.

Your handout specifies a 60-minute duration and requires before-and-after videos, updated code, and an exported chat history. <FileCite refid="turn0file0" linerangestart={2} line range_end={8} />

1. What exactly do you need to do?

<box gap={3}>

<row align=start gap={3}>

<AsyncImage query="GitHub repository code page screenshot dark theme" aspectRatio="3:2" width="104px"/>

<box flex="1" gap={1}>

Step 1 — Open the assigned repository

Visit [60numberguess on GitHub](<https://github.com/SETAPESU26/60numberguess>). Read the README.md file to understand the game's requirements and the features you need to implement.

</box>

</row>

<divider color="subtle"/>

<row align=start gap={3}>

<AsyncImage query="GitHub fork repository button interface screenshot" aspectRatio="3:2" width="104px"/>

<box flex="1" gap={1}>

Step 2 — Copy the project to your account

Fork the repository into your personal GitHub account, or clone it and push it to a new repository you own. Do not submit changes through a pull request to the original SETAPESU26 repository.

</box>

</row>

```
<divider color="subtle"/>
<row align=start gap={3}>
  <AsyncImage query="Python number guessing game terminal black window code output"
    aspectRatio="3:2" width="104px"/>
</box>
```

Step 3 — Run the original game

Download or clone the project, install any dependencies listed in its instructions, and run the Python code. Observe what works and what is broken. Record a 10-second video of the game **before making any changes**.

```
</box>
</row>
<divider color="subtle"/>
<row align=start gap={3}>
  <AsyncImage query="AI coding assistant writing Python code in Visual Studio Code editor"
    aspectRatio="3:2" width="104px"/>
</box>
```

Step 4 — Fix the broken code with AI

Use ChatGPT or another vibe-coding tool to inspect the code, identify bugs, and suggest fixes. Test each fix instead of blindly accepting generated code.

```
</box>
</row>
<divider color="subtle"/>
<row align=start gap={3}>
  <AsyncImage query="Python game source code in Visual Studio Code editor"
    aspectRatio="3:2" width="104px"/>
</box>
```

Step 5 — Implement all required features

Follow the README and add each listed feature individually. Keep a separate Git commit for each task, including the bug fix and the individual feature additions.

```
</box>
</row>
<divider color="subtle"/>
<row align=start gap={3}>
  <AsyncImage query="GitHub repository commits page source code files"
    aspectRatio="3:2" width="104px"/>
</box>
```

Step 6 — Record, push and submit

Record another 10-second video after all changes. Push the completed code to your own repository and upload the videos, updated code, and exported AI chat history into the Lab-4 folder of your lab repository.

```
</box>
</row>
</box>
```

These steps follow the sequence in your lab handout. <FileCite refid="turn0file0" linerange start={17} linerange_end={38} />

Avi, your Lab 4 is an **individual Vibe Coding assignment**. The goal is to take the Python game assigned to you on GitHub, understand it, fix its broken code using an AI tool like ChatGPT, implement the features listed in its README, and push the completed project to your own GitHub account.

Your handout specifies a 60-minute duration and requires before-and-after videos, updated code, and an exported chat history. <FileCite refid="turn0file0" linerangestart={2} linerange_end={8} />

1. What exactly do you need to do?

<box gap={3}>

<row align=start gap={3}>

<AsyncImage query="GitHub repository code page screenshot dark theme" aspectRatio="3:2" width="104px"/>

<box flex="1" gap={1}>

Step 1 — Open the assigned repository

Visit [60numberguess on GitHub](https://github.com/SETAPESU26/60numberguess). Read the README.md file to understand the game's requirements and the features you need to implement.

</box>

</row>

<divider color="subtle"/>

<row align=start gap={3}>

<AsyncImage query="GitHub fork repository button interface screenshot" aspectRatio="3:2" width="104px"/>

<box flex="1" gap={1}>

Step 2 — Copy the project to your account

Fork the repository into your personal GitHub account, or clone it and push it to a new repository you own. Do not submit changes through a pull request to the original SETAPESU26 repository.

</box>

</row>

<divider color="subtle"/>

<row align=start gap={3}>

<AsyncImage query="Python number guessing game terminal black window code output" aspectRatio="3:2" width="104px"/>

<box flex="1" gap={1}>

Step 3 — Run the original game

Download or clone the project, install any dependencies listed in its instructions, and run the Python code. Observe what works and what is broken. Record a 10-second video of the game **before making any changes**.

</box>

```

</row>
<divider color="subtle"/>
<row align=start gap={3}>
<AsyncImage query="AI coding assistant writing Python code in Visual Studio Code editor"
aspectRatio="3:2" width="104px"/>
<box flex="1" gap={1}>

```

Step 4 — Fix the broken code with AI

Use ChatGPT or another vibe-coding tool to inspect the code, identify bugs, and suggest fixes. Test each fix instead of blindly accepting generated code.

```

</box>
</row>
<divider color="subtle"/>
<row align=start gap={3}>
<AsyncImage query="Python game source code in Visual Studio Code editor"
aspectRatio="3:2" width="104px"/>
<box flex="1" gap={1}>

```

Step 5 — Implement all required features

Follow the README and add each listed feature individually. Keep a separate Git commit for each task, including the bug fix and the individual feature additions.

```

</box>
</row>
<divider color="subtle"/>
<row align=start gap={3}>
<AsyncImage query="GitHub repository commits page source code files"
aspectRatio="3:2" width="104px"/>
<box flex="1" gap={1}>

```

Step 6 — Record, push and submit

Record another 10-second video after all changes. Push the completed code to your own repository and upload the videos, updated code, and exported AI chat history into the Lab-4 folder of your lab repository.

```

</box>
</row>
</box>

```

These steps follow the sequence in your lab handout. <FileCite refid="turn0file0" linerange start={17} linerange_end={38} />

2. The exact tasks in YOUR GitHub repository

I checked the repository's README. It is a **Pygame-based Number Guessing game** with one deliberate bug and three features to implement. <Cite refs={["turn339325view0"]} />

```

<box border radius="lg" padding={3} gap={3}>
<row align=start gap={3}>
<box background="surface-secondary" radius="md" size="40px" align=center
justify=center>

```

```
<icon name="bug" size="xl"/>
```

```
</box>
```

```
<box flex="1" gap={1}>
```

Task 1 — Fix the empty input crash

If a player presses Submit without entering a number, the game currently crashes. Fix it so that it displays a warning, keeps running, and does not consume an attempt.

```
<badge color="danger">Bug fix</badge>
```

```
</box>
```

```
</row>
```

```
<divider color="subtle"/>
```

```
<row align="start" gap={3}>
```

```
<box background="surface-secondary" radius="md" size="40px" align="center"
justify="center">
```

```
<icon name="arrow-left-right" size="xl"/>
```

```
</box>
```

```
<box flex="1" gap={1}>
```

Task 2 — Display the narrowing search range

Track the minimum and maximum possible values based on previous guesses and display the updated range on screen.

Example: if the secret number is 63 and you guess 40, the game says TOO LOW! and narrows the possible range to 41-100.

```
</box>
```

```
</row>
```

```
<divider color="subtle"/>
```

```
<row align="start" gap={3}>
```

```
<box background="surface-secondary" radius="md" size="40px" align="center"
justify="center">
```

```
<icon name="history" size="xl"/>
```

```
</box>
```

```
<box flex="1" gap={1}>
```

Task 3 — Add recent guess history

Display a history panel showing previous guesses and color-coded hints.

Example: 40 – TOO LOW and 75 – TOO HIGH.

```
</box>
```

```
</row>
```

```
<divider color="subtle"/>
```

```
<row align="start" gap={3}>
```

```
<box background="surface-secondary" radius="md" size="40px" align="center"
justify="center">
```

```
<icon name="timer" size="xl"/>
```

```
</box>
```

```
<box flex="1" gap={1}>
```

Task 4 — Add a maximum attempt limit

Set a strict limit on the number of guesses. When the player runs out of attempts, show a Game Over screen revealing the secret number.

</box>

</row>

</box>

These are the four tasks explicitly listed in the repository README. <Cite refs={["turn339325view0"]} />

3. How to do the lab on your ASUS laptop

Follow these steps in order.

Step 1: Get the code

The easiest option is to use GitHub's Fork button on the repository page, then clone your copy.

Alternatively, open a terminal and run:

```
git clone https://github.com/SETAPESU26/60_number_guess.git
cd 60_number_guess
```

If you use this alternative, create a new repository under your personal GitHub account and connect it as your own remote before pushing.

Step 2: Install and run the game

Open the project folder in VS Code. In its terminal, run:

```
python --version
pip install pygame
python main.py
```

The README requires Python 3.10 or newer. If python doesn't work on Windows, try py instead. <Cite refs={["turn339325view0"]} />

Before editing anything, record a 10-second clip showing the original game and its empty-input crash. If the crash is difficult to capture, try it once to reproduce it, then record the demonstration.

Step 3: Use these prompts in ChatGPT

Give your AI coding assistant one task at a time. If you are using ChatGPT, upload the project files or paste the relevant code so it can inspect the actual implementation.

<box border radius="lg" padding={3} gap={3}>

<box gap={1}>

<row align=center>

<badge>Prompt 1</badge>

Fix the bug

<spacer/>

<button variant="ghost" size="sm" onClick={() => GenUI.copy("Inspect my existing Pygame Number Guessing project. Find and fix the bug where submitting an empty input

crashes the application. Show a warning message, keep the game running, and do not increment the attempt counter. Preserve the existing UI and code structure. Explain the changes and tell me how to test the fix.")]><icon name="copy" size="xs"/>
Copy</button>

</row>

<text color="secondary" size="sm">Test the empty input, a valid guess, and the Enter key before moving on.</text>

</box>

<divider color="subtle"/>

<box gap={1}>

<row align=center>

<badge>Prompt 2</badge>

Add dynamic range

<spacer/>

<button variant="ghost" size="sm" onClick={() => GenUI.copy("Implement a dynamic search range in my existing Number Guessing game. Start with the range 1-100. After each incorrect guess, update the minimum or maximum boundary based on whether the guess was too low or too high. Display the updated range on screen, preserve correct guessing behavior, and handle boundary values accurately. Explain and test the changes.")}><icon name="copy" size="xs"/> Copy</button>

</row>

</box>

<divider color="subtle"/>

<box gap={1}>

<row align=center>

<badge>Prompt 3</badge>

Add guess history

<spacer/>

<button variant="ghost" size="sm" onClick={() => GenUI.copy("Add a recent guess history panel to my existing Pygame Number Guessing game. Show the last several valid guesses with color-coded TOO LOW or TOO HIGH indicators. Update the history after each valid guess, prevent empty submissions from entering the history, and clear the history when a new game starts. Keep the UI readable and explain how to test it.")}><icon name="copy" size="xs"/> Copy</button>

</row>

</box>

<divider color="subtle"/>

<box gap={1}>

<row align=center>

<badge>Prompt 4</badge>

Add attempt limit

<spacer/>

<button variant="ghost" size="sm" onClick={() => GenUI.copy("Implement a maximum number of valid guesses in my existing Pygame Number Guessing game. Define a clear attempt limit, count only valid guesses, and show a Game Over screen revealing the secret

number when the limit is reached without a win. A correct guess on the final permitted attempt must still win. Prevent further guesses after the game ends and reset all game state when starting a new game. Explain and test the changes.")]><icon name="copy" size="xs"/> Copy</button>

</row>

</box>

</box>

Important: The README doesn't specify the exact attempt limit or how many guesses the history panel should retain. Choose sensible values, keep them consistent, and verify them with your tests.

Step 4: Test the completed game

Before pushing, check each item.

```
{@body const checks = ["Empty input shows a warning without crashing or using an attempt", "Valid guesses correctly show TOO LOW or TOO HIGH", "The displayed search range narrows correctly", "Recent guess history shows the correct directions", "The attempt limit triggers Game Over and reveals the number", "Guessing correctly displays the victory message", "Restarting with R resets the range, history and attempts", "No errors occur during normal gameplay"]}
```

```
{@body const [checked, setChecked] = DIL.useState([])}
```

```
<box border radius="lg" padding={3} gap={2}>
```

```
<row align=center>
```

```
<title size="md">Testing checklist</title>
```

```
<spacer/>
```

```
<text color="secondary" size="sm">{checked.length}/{checks.length}</text>
```

```
</row>
```

```
{#each checks as item, i}
```

```
<checkbox key={item} checked={checked.includes(i)} onChange={v => setChecked(old => v ? [...old, i] : old.filter(n => n !== i))} lineThrough={false}>{item}</checkbox>
```

```
{/each}
```

```
</box>
```

Step 5: Make separate Git commits

After completing and testing each task, save it as a separate commit. For example:

```
git add .
git commit -m "Fix empty input crash"
```

Repeat after each feature, changing the message accordingly:

- Implement dynamic search range
- Add recent guess history
- Add maximum attempts and game over

Then push the commits to your personal repository. If you're using a fork, push to your fork, not the original repository.

4. What you must submit

Your lab handout requires the following deliverables. <FileCite refid="turn0file0" linerange_start={33} linerange_end={38}/>

```
{@body const deliverables = [{title:"Before video",detail:"10 seconds showing the original game and broken behavior"},{title:"After video",detail:"10 seconds showing the fixed game and new features"},{title:"Updated code",detail:"Your completed project in your personal GitHub repository"},{title:"AI chat history",detail:"Export or provide the complete conversation as required by your instructor"}]}
```

```
<box border radius="lg" padding={3} gap={3}>
```

```
{#each deliverables as d}
```

```
<row align=start gap={2}>
```

```
<icon name="check-circle-2" size="lg" color="success"/>
```

```
<box flex="1" gap={1}>
```

```
{d.title}
```

```
<text color="secondary" size="sm">{d.detail}</text>
```

```
</box>
```

```
</row>
```

```
{/each}
```

```
</box>
```

The handout says to put the deliverables inside the Lab-4 folder of your lab repository. It also says not to raise a pull request to the original repository. <FileCite refid="turn0file0" linerangestart={17} linerange_end={38}/>

My recommendation: Start by forking the repository, running the untouched game, and recording the before video. Then complete the four tasks in order, testing and committing after each one.

If you upload the repository's main.py, game_engine.py, and text_box.py files here, I can walk you through the exact code changes for each task without guessing how the existing code is structured.

2. The exact tasks in YOUR GitHub repository

I checked the repository's README. It is a **Pygame-based Number Guessing game** with one deliberate bug and three features to implement. <Cite refs={["turn339325view0"]}/>

```
<box border radius="lg" padding={3} gap={3}>
```

```
<row align=start gap={3}>
```

```
<box background="surface-secondary" radius="md" size="40px" align=center justify=center>
```

```
<icon name="bug" size="xl"/>
```

```
</box>
```

```
<box flex="1" gap={1}>
```

Task 1 — Fix the empty input crash

If a player presses Submit without entering a number, the game currently crashes. Fix it so that it displays a warning, keeps running, and does not consume an attempt.

```
<badge color="danger">Bug fix</badge>
```

```
</box>
```

```

</row>
<divider color="subtle"/>
<row align=start gap={3}>
<box background="surface-secondary" radius="md" size="40px" align=center
justify=center>
<icon name="arrow-left-right" size="xl"/>
</box>
<box flex="1" gap={1}>

```

Task 2 — Display the narrowing search range

Track the minimum and maximum possible values based on previous guesses and display the updated range on screen.

Example: if the secret number is 63 and you guess 40, the game says T00 LOW! and narrows the possible range to 41-100.

```

</box>
</row>
<divider color="subtle"/>
<row align=start gap={3}>
<box background="surface-secondary" radius="md" size="40px" align=center
justify=center>
<icon name="history" size="xl"/>
</box>
<box flex="1" gap={1}>

```

Task 3 — Add recent guess history

Display a history panel showing previous guesses and color-coded hints.

Example: 40 – T00 LOW and 75 – T00 HIGH.

```

</box>
</row>
<divider color="subtle"/>
<row align=start gap={3}>
<box background="surface-secondary" radius="md" size="40px" align=center
justify=center>
<icon name="timer" size="xl"/>
</box>
<box flex="1" gap={1}>

```

Task 4 — Add a maximum attempt limit

Set a strict limit on the number of guesses. When the player runs out of attempts, show a Game Over screen revealing the secret number.

```

</box>
</row>
</box>

```

These are the four tasks explicitly listed in the repository README. <Cite
 refs={["turn339325view0"]}/>

3. How to do the lab on your ASUS laptop

Follow these steps in order.

Step 1: Get the code

The easiest option is to use GitHub's Fork button on the repository page, then clone your copy.

Alternatively, open a terminal and run:

```
git clone https://github.com/SETAPESU26/60_number_guess.git
cd 60_number_guess
```

If you use this alternative, create a new repository under your personal GitHub account and connect it as your own remote before pushing.

Step 2: Install and run the game

Open the project folder in VS Code. In its terminal, run:

```
python --version
pip install pygame
python main.py
```

The README requires Python 3.10 or newer. If python doesn't work on Windows, try py instead. <Cite refs={["turn339325view0"]} />

Before editing anything, record a 10-second clip showing the original game and its empty-input crash. If the crash is difficult to capture, try it once to reproduce it, then record the demonstration.

Step 3: Use these prompts in ChatGPT

Give your AI coding assistant one task at a time. If you are using ChatGPT, upload the project files or paste the relevant code so it can inspect the actual implementation.

<box border radius="lg" padding={3} gap={3}>

<box gap={1}>

<row align=center>

<badge>Prompt 1</badge>

Fix the bug

<spacer/>

<button variant="ghost" size="sm" onClick={() => GenUI.copy("Inspect my existing Pygame Number Guessing project. Find and fix the bug where submitting an empty input crashes the application. Show a warning message, keep the game running, and do not increment the attempt counter. Preserve the existing UI and code structure. Explain the changes and tell me how to test the fix.")}><icon name="copy" size="xs"/>

Copy</button>

</row>

<text color="secondary" size="sm">Test the empty input, a valid guess, and the Enter key before moving on.</text>

</box>

<divider color="subtle"/>

```
<box gap={1}>
<row align=center>
<badge>Prompt 2</badge>
```

Add dynamic range

```
<spacer/>
<button variant="ghost" size="sm" onClick={() => GenUI.copy("Implement a dynamic
search range in my existing Number Guessing game. Start with the range 1-100. After
each incorrect guess, update the minimum or maximum boundary based on whether the
guess was too low or too high. Display the updated range on screen, preserve correct
guessing behavior, and handle boundary values accurately. Explain and test the
changes.")}><icon name="copy" size="xs"/> Copy</button>
</row>
</box>
```

```
<divider color="subtle"/>
<box gap={1}>
<row align=center>
<badge>Prompt 3</badge>
```

Add guess history

```
<spacer/>
<button variant="ghost" size="sm" onClick={() => GenUI.copy("Add a recent guess
history panel to my existing Pygame Number Guessing game. Show the last several valid
guesses with color-coded TOO LOW or TOO HIGH indicators. Update the history after each
valid guess, prevent empty submissions from entering the history, and clear the history
when a new game starts. Keep the UI readable and explain how to test it.")}><icon
name="copy" size="xs"/> Copy</button>
</row>
</box>
```

```
<divider color="subtle"/>
<box gap={1}>
<row align=center>
<badge>Prompt 4</badge>
```

Add attempt limit

```
<spacer/>
<button variant="ghost" size="sm" onClick={() => GenUI.copy("Implement a maximum
number of valid guesses in my existing Pygame Number Guessing game. Define a clear
attempt limit, count only valid guesses, and show a Game Over screen revealing the secret
number when the limit is reached without a win. A correct guess on the final permitted
attempt must still win. Prevent further guesses after the game ends and reset all game
state when starting a new game. Explain and test the changes.")}><icon name="copy"
size="xs"/> Copy</button>
</row>
</box>
</box>
```

Important: The README doesn't specify the exact attempt limit or how many guesses the history panel should retain. Choose sensible values, keep them consistent, and verify them

with your tests.

Step 4: Test the completed game

Before pushing, check each item.

```
{@body const checks = ["Empty input shows a warning without crashing or using an attempt", "Valid guesses correctly show TOO LOW or TOO HIGH", "The displayed search range narrows correctly", "Recent guess history shows the correct directions", "The attempt limit triggers Game Over and reveals the number", "Guessing correctly displays the victory message", "Restarting with R resets the range, history and attempts", "No errors occur during normal gameplay"]}

{@body const [checked, setChecked] = DIL.useState([])}

<box border radius="lg" padding={3} gap={2}>
  <row align=center>
    <title size="md">Testing checklist</title>
    <spacer/>
    <text color="secondary" size="sm">{checked.length}/{checks.length}</text>
  </row>
  {#each checks as item, i}
    <checkbox key={item} checked={checked.includes(i)} onChange={v => setChecked(old => v ? [...old, i] : old.filter(n => n !== i))} lineThrough={false}>{item}</checkbox>
  {/each}
</box>
```

Step 5: Make separate Git commits

After completing and testing each task, save it as a separate commit. For example:

```
git add .
git commit -m "Fix empty input crash"
```

Repeat after each feature, changing the message accordingly:

- Implement dynamic search range
- Add recent guess history
- Add maximum attempts and game over

Then push the commits to your personal repository. If you're using a fork, push to your fork, not the original repository.

4. What you must submit

Your lab handout requires the following deliverables. <FileCite refid="turn0file0" linerange start={33} linerange_end={38}/>

```
{@body const deliverables = [{title:"Before video",detail:"10 seconds showing the original game and broken behavior"},{title:"After video",detail:"10 seconds showing the fixed game and new features"},{title:"Updated code",detail:"Your completed project in your personal GitHub repository"},{title:"AI chat history",detail:"Export or provide the complete conversation as required by your instructor"}]}

<box border radius="lg" padding={3} gap={3}>
  {#each deliverables as d}
```

```

<row align=start gap={2}>
<icon name="check-circle-2" size="lg" color="success"/>
<box flex="1" gap={1}>
{d.title}
<text color="secondary" size="sm">{d.detail}</text>
</box>
</row>
{/each}
</box>

```

The handout says to put the deliverables inside the Lab-4 folder of your lab repository. It also says not to raise a pull request to the original repository. <FileCite refid="turn0file0" linerangestart={17} linerange_end={38}/>

My recommendation: Start by forking the repository, running the untouched game, and recording the before video. Then complete the four tasks in order, testing and committing after each one.

If you upload the repository's main.py, game_engine.py, and text_box.py files here, I can walk you through the exact code changes for each task without guessing how the existing code is structured.

User

tell me what to do step by step and ill do it

ChatGPT

Alright Avi, we'll do this **one step at a time**. You do each step on your laptop, send me a screenshot if needed, and I'll tell you exactly what to do next.

We'll complete the lab in the required order so you don't miss any deliverables.

Step 1 — Open your assigned GitHub repository

```

<box gap={3}>
<row align=start gap={3}>
<AsyncImage query="GitHub repository page showing repository files README.md and
green Code button" aspectRatio="4:3" width="124px"/>
<box flex="1" gap={1}>

```

1. Open the repository

Click the link below to open your assigned project.

[Open 60numberguess on GitHub](<https://github.com/SETAPESU26/60numberguess>)

```

</box>
</row>
<row align=start gap={3}>
<AsyncImage query="GitHub README markdown rendered repository page"
aspectRatio="4:3" width="124px"/>
<box flex="1" gap={1}>

```

2. Find the README

Scroll down on the repository page until you see README.md. Read the instructions and the list of required tasks.

</box>

</row>

</box>

Don't edit or download anything yet. First, we need to make sure you're looking at the correct repository.

When you've opened the page, **send me a screenshot of what you see.** I'll guide you through Step 2: copying the repository into your own GitHub account.

User

Number Guessing Repair Lab

This project is an interactive number deduction game using Pygame. It introduces students to string-to-integer parsing safety, boundary and input sanitization, dynamic range feedback, and text-input UI widgets within an object-oriented codebase.

What's Provided

A working Number Guessing game with:

- A secret number generated randomly between 1 and 100 on game initialization and resets
- A custom TextBox input component handling digit typing, backspace, and active selection states
- Interactive guess submission via the Return / Enter key or clicking the SUBMIT button
- Dynamic hint feedback (T00 LOW!, T00 HIGH!, or CORRECT!) along with an attempt counter
- Win state handling and restart capability via the R key

It has **one deliberate bug** and **three optional features** left as tasks to implement. You are expected to **analyze, interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

pip install pygame

3. Run the game:

python main.py

Controls: Type digits into the text box and press Return (or click SUBMIT) to guess. Press R to start a new game after winning.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the empty input crash bug

Submitting a guess without typing any numbers causes the application to crash immediately with a value parsing error. Ensure that empty submissions are safely handled with a warning prompt without terminating the program or consuming an attempt.

Task 2: Implement dynamic search range display

The game currently only provides one-off high or low indicators without keeping track of the narrowing search window. Track the valid minimum and maximum boundaries established by previous guesses and display the narrowed range on screen to guide the player's next move

Task 3: Implement Recent Guess History Tracker

Players have no visual record of previous numbers they have already tested. Add a history log panel displaying the last several guesses alongside color-coded directional indicators showing whether each was too high or too low.

Task 4: Implement maximum Attempts Constraint & Failure State

Players currently enjoy unlimited guesses, removing stakes from deduction. Introduce a strict attempt limit that triggers a Game Over screen revealing the hidden number if the player runs out of tries.

Expected Behavior

- Typing numbers into the text box and pressing Return or clicking SUBMIT registers a guess.
- Submitting an empty input box displays a warning message without crashing the game.
- The game accurately indicates whether the guess is too high or too low, incrementing the attempt count each time.
- Guessing the exact number displays the victory message and unlocks the R key restart option.

Folder Structure

```
number_guess/  
├── game/  
│   ├── game_engine.py  
│   └── text_box.py  
└── main.py
```


Submission Checklist

Submission is only the following three things:

- ☐ A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- ☐ A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- ☐ The Chat/LLM used page link, with the complete chat history

ChatGPT

Perfect, Avi. Now we know exactly what your lab requires. We'll do it together, one step at a time. **Don't jump ahead — complete each step and tell me when you're done.**

Step 1 — Fork the repository to your GitHub account

First, we need to create your own copy of the assigned project so your changes don't affect the original repository.

```
<box gap={3}>
```

```
<row align=start gap={3}>
```

```
<AsyncImage query="GitHub repository top toolbar Fork button beside Star and Watch buttons" aspectRatio="5:3" width="120px"/>
```

```
<box flex="1" gap={1}>
```

1. Open your assigned repository

[Click here to open 60numberguess](https://github.com/SETAPESU26/60numberguess)

```
</box>
```

```
</row>
```

```
<row align=start gap={3}>
```

```
<AsyncImage query="GitHub fork repository dialog owner dropdown create fork button" aspectRatio="5:3" width="120px"/>
```

```
<box flex="1" gap={1}>
```

2. Click `Fork`

Look near the top-right of the repository page. Select your personal GitHub account as the owner.

```
</box>
```

```
</row>
```

```
<row align=start gap={3}>
```

```
<AsyncImage query="GitHub personal repository page showing repository name and Code button" aspectRatio="5:3" width="120px"/>
```

```
<box flex="1" gap={1}>
```

3. Create the fork

Click **Create fork**. Wait until GitHub opens the copied repository under your account.

```
</box>
```

```
</row>
```

</box>

What you should see: The repository URL should now start with:

`https://github.com/YOUR-USERNAME/60_number_guess`

Your actual username will replace YOUR-USERNAME.

Your turn

Do Step 1 now. Once your fork is created, **send me a screenshot of the new repository page.**

Then I'll guide you through Step 2: downloading the code and running the original game so you can record the required 10-second *before* video.

User

should i not clone it

ChatGPT

Yes, Avi! **You should clone it to your laptop, but first fork it to your GitHub account.** They serve two different purposes.

- **Fork** = creates your own copy of the repository on GitHub.
- **Clone** = downloads that repository onto your laptop so you can edit and run the code.

Let's do both in order.

Step 1 — Fork the repository

1. Open [60numberguess](https://github.com/SETAPESU26/60numberguess).
2. Click **Fork** in the top-right corner.
3. Select your GitHub account and click **Create fork**.

Step 2 — Clone your fork

Once your fork is created, open its page and click the green **Code** button.

<AsyncImage query="GitHub repository green Code dropdown Local HTTPS clone URL Copy button" aspectRatio="5:3" width="100%"/>

1. Select **HTTPS**.
2. Click the copy icon beside the repository URL.
3. Open a terminal on your laptop (PowerShell or the VS Code terminal).
4. Run these commands, replacing the URL with the one you copied:

```
git clone YOUR_COPIED_URL
cd 60_number_guess
```

Step 3 — Tell me when you're done

Once the clone finishes, send me a screenshot of your terminal.

Then we'll run the original game and record the **10-second before video before changing any code**, as your lab requires. □

User

done

ChatGPT

□ Great! Now let's run the original game before making any changes.

Step 2 — Open the project in VS Code

1. Open VS Code.

2. Open your cloned project folder:

- Click **File → Open Folder**.
- Navigate to the 60_number_guess folder you cloned.
- Select the folder and click **Select Folder**.

3. Open the terminal in VS Code using **Terminal → New Terminal**.

Step 3 — Check Python and install Pygame

Run these commands **one at a time** in the terminal.

First:

```
python --version
```

You should see Python 3.10 or newer.

Next:

```
python -m pip install pygame
```

Wait for the installation to finish.

Step 4 — Run the original game

Run:

```
python main.py
```

A game window should open.

- Try typing a number and submitting it.
- Then clear the input box and press **Enter** or click **SUBMIT**.
- The empty-input bug should cause an error or crash, which is what we need to demonstrate before fixing it.

Important: Don't modify any code yet.

Step 5 — Record your before video

Use Win + Alt + R to start and stop recording with Xbox Game Bar, if enabled. Record about 10 seconds showing the original game and the broken behavior.

If the game crashes too quickly to show clearly, record the game running first, then reproduce the bug.

Your turn: Run `python --version` in the VS Code terminal and tell me what it prints. We'll handle any errors before moving forward.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file  
000000007f288210b58b819174fac02b?sharedconversation  
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size  
bytes":76499,"width":1762,"height":896,"metadata":{"sanitized":true}}
```

ChatGPT

You're in the right window, Avi! The issue is that Windows is searching your Desktop, and it cannot find the project folder there.

Let's fix this.

Step 1 — Close this window

Click **Cancel** at the bottom-right.

Step 2 — Open VS Code's terminal

In VS Code, click **Terminal → New Terminal** at the top.

Copy and run this command:

```
cd $HOME\Desktop  
dir 60_number_guess
```

If the folder is on your Desktop, this should display its contents.

If it says the folder doesn't exist

Try this:

```
cd $HOME\Downloads  
dir 60_number_guess
```

Step 3 — Open the correct folder

Once you find the folder, run the appropriate command. For example, if it is on your Desktop:

```
code "$HOME\Desktop\60_number_guess"
```

If it is in Downloads:

```
code "$HOME\Downloads\60_number_guess"
```

If the code command doesn't work, we'll use VS Code's **File → Open Folder** and navigate to the location you found.

Your turn: First click Cancel, open the terminal, and run the two Desktop commands. Send me a screenshot of the terminal output, and I'll tell you exactly what to do next.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file  
00000000e8e08210a70a5eaf13b66f60?sharedconversation  
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size  
bytes":86341,"width":1966,"height":486,"metadata":{"sanitized":true}}
```

ChatGPT

No worries, Avi. ☐ The screenshot shows that the project isn't on your Desktop. We'll locate it.

Step 1 — Check your Downloads folder

In the same VS Code terminal, copy and run:

```
cd $HOME\Downloads
dir 60_number_guess
```

- **If you see files or folders listed:** Great! We've found it.
- **If you get another error:** Continue to Step 2 below.

Step 2 — Search your user folder

Run this command:

```
Get-ChildItem $HOME -Directory -Recurse -Filter "60_number_guess" -ErrorAction SilentlyContinue
```

This searches your user folder for the cloned project. It may take a little while.

Step 3 — Tell me what happened

Send me a screenshot of the output.

If nothing appears, don't worry. We'll clone the repository again into a location that's easy to find, such as your Desktop, and continue from there.

User

```
{"contentType":"imageassetpointer","assetpointer":"sediment://file
000000000aa08210b1833908027587bc?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":140202,"width":2048,"height":254,"metadata":{"sanitized":true}}
```

ChatGPT

Perfect, Avi! We found your project folder. ☐

Your screenshot shows it is located at:

```
C:\Users\avane\60_number_guess
```

Step 1 — Open the project in VS Code

In the same terminal, copy and run:

```
cd "$HOME\60_number_guess"
code .
```

This should open the correct project folder in VS Code.

Step 2 — Check the project files

In VS Code, look at the **Explorer panel on the left**. You should see files and folders such as:

- main.py
- README.md

- game
- game_engine.py
- text_box.py

Step 3 — Run the original game

In the terminal, run:

```
python --version
```

Then run:

```
python -m pip install pygame
```

Finally:

```
python main.py
```

Don't edit any files yet. We need to run the original game and record your 10-second before video first.

Tell me what happens when you run code . and then python main.py.

User

```
{"contentType":"imageassetpointer","assetpointer":"sediment://file
00000000737c8210bcd383f82905f3de?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":126385,"width":2048,"height":715,"metadata":{"sanitized":true}}
```

sidenote- this random terminal is opening again and again tell me what to do to stop it

ChatGPT

Avi, that window is **Windows Subsystem for Linux (WSL)** trying to install or update itself. You don't need WSL for this Python lab; we can use Windows PowerShell instead.

1. Stop the random terminal for now

1. Press **Esc** in that black window to cancel the installation prompt.
2. Close the window using the **X** at the top-right.

2. Stop it from opening repeatedly

Let's first check whether Windows is launching it automatically.

1. Press Ctrl + Shift + Esc to open **Task Manager**.
2. Click **Startup apps** in the left sidebar.
3. Look for entries named **WSL, Windows Subsystem for Linux**, or a terminal-related app.
4. If you find an entry clearly responsible for launching WSL, right-click it and choose **Disable**.

Don't disable Windows Terminal or unrelated system processes.

3. Open your project without WSL

Go back to VS Code and use its existing **PowerShell** terminal. Run:

```
cd "$HOME\60_number_guess"
```

Then check that your project files are there:

```
dir
```

You should see `main.py` and the `game` folder.

Don't run ``code .`` again yet—it appears to have triggered the WSL window on your system, and we can open the folder directly through VS Code instead.

Tell me one thing

Does the black WSL window appear **only when you run ``code .``**, or does it pop up randomly even when you're not using VS Code? That will help me give you the correct permanent fix.

User

the game is running what do i do next

ChatGPT

Great, Avi! Now we need to record the original game before changing any code. Follow these steps in order.

Step 1 — Record the BEFORE video

<text color="secondary" size="sm">Required by your lab: 10 seconds of gameplay before making changes.</text>

1. Keep the game window open.
2. Press **Win + Alt + R** to start recording with Xbox Game Bar.
3. Type a number into the game and click **SUBMIT** to show that it works.
4. Clear the input box and click **SUBMIT** again to reproduce the empty-input bug.
5. Stop recording with **Win + Alt + R** after about 10 seconds.

If Xbox Game Bar doesn't record the game, tell me and I'll give you another method.

Step 2 — Confirm the bug

After recording, try submitting an empty input once more.

- Does the game crash or close?
- Or does it display a warning and continue running?

Your README says the empty submission is supposed to crash the original game, so we need to verify what happens in your version.

Step 3 — Send me a screenshot

Once you've recorded the video, send me a screenshot of the game window or the error you get.

Don't edit the code yet. Next, we'll fix Task 1, test it, and save a separate Git commit before moving on to the three new features.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file0000000040948210bc231a7a7a0c19aa?sharedconversation
```

```
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":94329,"width":1560,"height":520,"metadata":{"sanitized":true}}}
```

so when i press enter without any input the game crashes and closes

ChatGPT

Perfect, Avi! This confirms that **Task 1 is reproducible**. The error `ValueError: invalid literal for int() with base 10: ''` occurs because the game tries to convert an empty string into an integer.

Now we'll fix it using AI, test it, and commit the change.

Step 1 — Open `game_engine.py`

In VS Code's Explorer panel on the left:

1. Expand the game folder.
2. Click `game_engine.py`.
3. Find the `submit_guess()` function. Your error indicates the problematic line is around line 26:

```
guess = int(self.input_box.text)
```

Don't change anything yet. We want to inspect the surrounding function first.

Step 2 — Send me the function

Copy and paste the entire `submit_guess()` function here. Include its indentation and all the lines until the next function begins.

I'll give you the exact corrected code for your existing implementation, so we don't accidentally break the game.

Step 3 — After the fix

We'll do the following:

1. Handle empty input with a warning message.
2. Ensure an empty submission doesn't increase the attempt counter.
3. Run `python main.py` again and verify that the game stays open.
4. Save a separate Git commit named `Fix empty input crash`.

Then we'll implement the remaining three features one at a time.

User

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)
        self.attempts = 0
```



```

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False

self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

self.font_title = pygame.font.SysFont(None, 42)
self.font_medium = pygame.font.SysFont(None, 28)
self.font_btn = pygame.font.SysFont(None, 26)

def submit_guess(self):
    if self.game_won:
        return

    guess = int(self.input_box.text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)
    elif guess > self.secret_number:
        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)
    else:
        self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
        self.feedback_color = (80, 220, 90)
        self.game_won = True

def reset(self):
    self.secret_number = random.randint(1, 100)
    self.attempts = 0
    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.input_box.clear()

def handle_event(self, event):
    self.inputbox.handleevent(event)

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RETURN:
            self.submit_guess()
        elif event.key == pygame.Kr and self.gamewon:
            self.reset()

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):

```

```

self.submit_guess()

def update(self):
    pass

def render(self, screen):
    screen.fill((30, 34, 42))

    titlesurf = self.fonttitle.render("Number Guessing Arena", True, (245, 245, 245))
    screen.blit(titlesurf, (self.width // 2 - titlesurf.get_width() // 2, 35))

    attemptssurf = self.fontmedium.render(f"Attempts: {self.attempts}", True, (180, 185, 195))
    screen.blit(attemptssurf, (self.width // 2 - attemptssurf.get_width() // 2, 95))
    self.input_box.render(screen)

    pygame.draw.rect(screen, (50, 150, 80), self.submitbtn, borderradius=6)
    pygame.draw.rect(screen, (220, 220, 220), self.submitbtn, width=2, borderradius=6)
    btntext = self.fontbtn.render("SUBMIT", True, (255, 255, 255))
    screen.blit(
        btn_text,
        (self.submitbtn.centerx - btntext.getwidth() // 2, self.submitbtn.centery - btntext.get
        height() // 2),
    )

    feedbacksurf = self.fontmedium.render(self.feedbackmsg, True, self.feedbackcolor)
    screen.blit(feedbacksurf, (self.width // 2 - feedbacksurf.get_width() // 2, 235))

    if self.game_won:
        restartsurf = self.fontmedium.render("Press [R] to Start a New Game", True, (255, 220, 80))
        screen.blit(restartsurf, (self.width // 2 - restartsurf.get_width() // 2, 295))

this is game_engine.py

```

ChatGPT

Great, Avi. I've got your game_engine.py. We'll fix **Task 1: Empty input crash** first, without changing the rest of the game.

Step 1 — Replace the submit_guess() function

In VS Code, open game/game_engine.py.

Find the existing def submit_guess(self): function and replace that entire function with this code.

<CodeBlock language="python" editable>

```

def submit_guess(self):
    if self.game_won:
        return

    # Prevent the game from crashing on empty input
    if not self.input_box.text.strip():

```

```

self.feedback_msg = "Please enter a number!"
self.feedback_color = (255, 220, 80)
return

guess = int(self.input_box.text)

self.attempts += 1
self.input_box.clear()

if guess < self.secret_number:
self.feedback_msg = f"TOO LOW! (Guess was {guess})"
self.feedback_color = (80, 160, 240)
elif guess > self.secret_number:
self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
self.feedback_color = (240, 100, 80)
else:
self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
self.feedback_color = (80, 220, 90)
self.game_won = True
</CodeBlock>

```

What changed? We added a check before `int()` converts the input to an integer. If the box is empty, the game displays a warning and returns immediately. This means the game won't crash, and the attempt count won't increase.

Step 2 — Save and run the game

1. Press `Ctrl + S` to save.
2. In the VS Code terminal, run:

```
python main.py
```

3. Click **SUBMIT** with the input box empty.
4. Check that:
 - The game stays open.
 - Please enter a number! appears.
 - The attempt count stays at 0.
5. Enter a number and submit it to confirm normal guessing still works.

Step 3 — Tell me the result

Did the game stay open and show the warning when you submitted an empty input?

Don't move to the next feature yet. Once this works, we'll create the separate Git commit for Task 1, then implement the dynamic search range.

User

yes its fixed whats next

ChatGPT

Nice, Avi! ☐ Task 1 is fixed. Next, we'll save it as a separate Git commit, as your lab requires, and then move to Task 2.

Step 1 — Commit the bug fix

In your VS Code terminal, run these commands one at a time:

1. Check which files changed

```
git status
```

2. Stage the file you changed

```
git add game/game_engine.py
```

3. Create the commit

```
git commit -m "Fix empty input crash"
```

You should see a message confirming that the commit was created.

Step 2 — Implement Task 2: Dynamic search range

Currently, the game tells you whether your guess is too high or too low. We want it to also show the range in which the secret number must lie.

For example, suppose the secret number is 63.

- You guess 40 → TOO LOW! and the range becomes **41-100**.
- You guess 80 → TOO HIGH! and the range becomes **41-79**.
- You guess 63 → CORRECT!

We'll update the game to track and display this range.

First, make this change in `__init__`

Open `game/game_engine.py`. Inside `def __init__(self, width, height):`, find:

```
self.attempts = 0
```

Immediately below it, add:

```
self.min_range = 1
self.max_range = 100
```

Next, update `submit_guess()`

Find these existing lines:

```
if guess < self.secret_number:
    self.feedback_msg = f"TOO LOW! (Guess was {guess})"
```

Change the low-guess branch to:

```
if guess < self.secret_number:
    self.min_range = max(self.min_range, guess + 1)
    self.feedback_msg = f"TOO LOW! (Guess was {guess})"
```

Then find the high-guess branch and update it to:

```
elif guess > self.secret_number:
    self.max_range = min(self.max_range, guess - 1)
    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
```

Finally, display the range

Inside `render()`, find the line that renders the attempts counter:

```
attempts_surf = self.font_medium.render(f"Attempts: {self.attempts}", True, (180, 185, 195))
```

Immediately after the two lines that render and display that counter, add:

```
range_surf = self.font_medium.render(
    f"Possible range: {self.min_range} - {self.max_range}",
    True,
    (180, 185, 195)
)
screen.blit(
    range_surf,
    (self.width // 2 - range_surf.get_width() // 2, 120)
)
```

The input box starts at vertical position 150, so this places the range above it.

Reset the range for a new game

Inside `reset()`, immediately after `self.attempts = 0`, add:

```
self.min_range = 1
self.max_range = 100
```

Step 3 — Test Task 2

Save with `Ctrl + S`, then run:

```
python main.py
```

Check that:

- The initial range is 1 - 100.
- A low guess increases the minimum.
- A high guess decreases the maximum.
- Pressing R after winning resets the range to 1 - 100.

Once you've tested it, tell me whether the range updates correctly. Then we'll commit Task 2 separately and move on to the guess history feature.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file
00000000a5d882109718a88eccf4ba01?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":59028,"width":1408,"height":398,"metadata":{"sanitized":true}}
```

ChatGPT

Avi, this is good! Your screenshot shows that Git is working and your changes to `game/game_engine.py` are detected. The file is **modified but not committed yet**.

Let's finish the commit for Task 1 before adding the next feature.

Step 1 — Commit the bug fix

Run these commands in your VS Code terminal, one at a time:

1. Stage the modified file

```
git add game/game_engine.py
```

2. Commit Task 1

```
git commit -m "Fix empty input crash"
```

3. Verify the commit

```
git log -1 --oneline
```

You should see a commit message containing `Fix empty input crash`.

Step 2 — Push your commit to GitHub

Run:

```
git push origin main
```

This pushes to the remote currently named `origin`. Your screenshot shows that `origin/main` is configured, but it doesn't establish whether `origin` points to your fork or the original repository.

Before pushing, verify the destination with:

```
git remote -v
```

Make sure the URL belongs to **your personal GitHub account**, not `SETAPESU26`. If it points to the original repository, don't push yet; send me the output and we'll correct it.

Step 3 — Tell me when done

Once Task 1 is committed and pushed, reply **done**. Then we'll implement Task 2, the dynamic search range, and test it before creating its own separate commit.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file
00000000d1288210ab912c3e558ccfb1?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":60816,"width":1224,"height":308,"metadata":{"sanitized":true}}
```

ChatGPT

Perfect, Avi! ☐ Task 1 is now committed and pushed to your personal GitHub repository:

https://github.com/Avi2106/60_number_guess

Your screenshot confirms that the push succeeded.

Step 2 — Implement the dynamic search range

Now we'll add Task 2. The game should narrow the possible range after every incorrect guess.

For example, if the secret number is 63:

- Guess 40 → range becomes 41–100
- Guess 80 → range becomes 41–79
- Guess 63 → you win!

1. Open `game/game_engine.py`

We'll make three small changes to the existing code.

2. Add the range variables

Inside `__init__()`, find:

```
self.attempts = 0
```

Immediately below it, add:

```
self.min_range = 1
self.max_range = 100
```

3. Update the low and high guess logic

Inside `submit_guess()`, find the `if guess < self.secret_number:` branch and add the range update as shown:

```
if guess < self.secret_number:
    self.min_range = max(self.min_range, guess + 1)
    self.feedback_msg = f"TOO LOW! (Guess was {guess})"
    self.feedback_color = (80, 160, 240)
elif guess > self.secret_number:
    self.max_range = min(self.max_range, guess - 1)
    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)
```

Keep the existing `else` branch for the correct guess unchanged.

4. Display the range on the screen

Inside `render()`, find the line:

```
screen.blit(attempts_surf, (self.width // 2 - attempts_surf.get_width() // 2, 95))
```

Immediately below it, add:

```
range_surf = self.font_medium.render(
    f"Possible range: {self.min_range} - {self.max_range}",
    True,
    (180, 185, 195)
)
screen.blit(
    range_surf,
    (self.width // 2 - range_surf.get_width() // 2, 120)
)
```

This displays the current range between the attempts counter and the input box.

5. Reset the range when restarting

Inside `reset()`, find:

```
self.attempts = 0
```

Immediately below it, add:

```
self.min_range = 1
self.max_range = 100
```

6. Save and test

Press Ctrl + S, then run:

```
python main.py
```

Verify that the range starts at 1 - 100, narrows correctly after high and low guesses, and resets to 1 - 100 when you restart the game.

Your turn: Make these changes and run the game. Tell me whether the range displays and updates correctly. Then we'll create the separate Task 2 commit and move to guess history.

User

give me the game engine py file with the changes

ChatGPT

Here is the complete updated game_engine.py, Avi. It includes **Task 1 (empty input crash fix)** and **Task 2 (dynamic search range)** while preserving your existing game logic.

Replace all the code in game/game_engine.py with this.

```
<CodeBlock language="python" editable>
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)
        self.attempts = 0

        # Task 2: Track the possible range
        self.min_range = 1
        self.max_range = 100

        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)
        self.game_won = False

        self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
        self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

        self.font_title = pygame.font.SysFont(None, 42)
        self.font_medium = pygame.font.SysFont(None, 28)
        self.font_btn = pygame.font.SysFont(None, 26)

    def submit_guess(self):
        if self.game_won:
            return

        # Task 1: Handle empty input safely
        if not self.input_box.text.strip():
```



```

self.feedback_msg = "Please enter a number!"
self.feedback_color = (255, 220, 80)
return

guess = int(self.input_box.text)
self.attempts += 1
self.input_box.clear()

if guess < self.secret_number:
    # Task 2: Narrow the minimum possible value
    self.minrange = max(self.minrange, guess + 1)
    self.feedback_msg = f"TOO LOW! (Guess was {guess})"
    self.feedback_color = (80, 160, 240)

elif guess > self.secret_number:
    # Task 2: Narrow the maximum possible value
    self.maxrange = min(self.maxrange, guess - 1)
    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)

else:
    self.feedback_msg = (
        f"CORRECT! Found in {self.attempts} attempts."
    )
    self.feedback_color = (80, 220, 90)
    self.game_won = True

def reset(self):
    self.secret_number = random.randint(1, 100)
    self.attempts = 0

    # Reset the search range for a new game
    self.min_range = 1
    self.max_range = 100

    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.input_box.clear()

def handle_event(self, event):
    self.inputbox.handleevent(event)

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RETURN:
            self.submit_guess()
        elif event.key == pygame.Kr and self.gamewon:
            self.reset()

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:

```

```

if self.submit_btn.collidepoint(event.pos):
    self.submit_guess()
def update(self):
    pass
def render(self, screen):
    screen.fill((30, 34, 42))
    titlesurf = self.fonttitle.render(
        "Number Guessing Arena", True, (245, 245, 245)
    )
    screen.blit(
        title_surf,
        (self.width // 2 - titlesurf.getwidth() // 2, 35),
    )
    attemptssurf = self.fontmedium.render(
        f"Attempts: {self.attempts}", True, (180, 185, 195)
    )
    screen.blit(
        attempts_surf,
        (self.width // 2 - attemptssurf.getwidth() // 2, 95),
    )
    # Task 2: Display the current possible search range
    rangesurf = self.fontmedium.render(
        f"Possible range: {self.minrange} - {self.maxrange}",
        True,
        (180, 185, 195),
    )
    screen.blit(
        range_surf,
        (self.width // 2 - rangesurf.getwidth() // 2, 120),
    )
    self.input_box.render(screen)
    pygame.draw.rect(
        screen, (50, 150, 80), self.submitbtn, borderradius=6
    )
    pygame.draw.rect(
        screen, (220, 220, 220), self.submit_btn,
        width=2, border_radius=6
    )
    btntext = self.fontbtn.render(
        "SUBMIT", True, (255, 255, 255)
    )

```

```

screen.blit(
    btn_text,
    (
        self.submitbtn.centerx - btntext.get_width() // 2,
        self.submitbtn.centery - btntext.get_height() // 2,
    ),
)

feedbacksurf = self.fontmedium.render(
    self.feedbackmsg, True, self.feedbackcolor
)

screen.blit(
    feedback_surf,
    (
        self.width // 2 - feedbacksurf.getwidth() // 2,
        235,
    ),
)

if self.game_won:
    restartsurf = self.fontmedium.render(
        "Press [R] to Start a New Game",
        True,
        (255, 220, 80),
    )

    screen.blit(
        restart_surf,
        (
            self.width // 2 - restartsurf.getwidth() // 2,
            295,
        ),
    )

```

</CodeBlock>

Next steps

1. Save the file with Ctrl + S.
2. Run `python main.py` in your terminal.
3. Verify the initial range is 1 - 100.
4. Enter a low guess and confirm the minimum increases; enter a high guess and confirm the maximum decreases.
5. Confirm that an empty submission still displays a warning without crashing.

Once Task 2 works, run these commands to commit it separately:

```

git add game/game_engine.py
git commit -m "Implement dynamic search range"
git push origin main

```

Important: Your current file contains only Tasks 1 and 2. We'll add guess history and the maximum-attempt Game Over state afterward, each as separate tasks and commits.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file
00000000708882109d60e99bc5ec374a?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":34910,"width":1216,"height":782,"metadata":{"sanitized":true}}
how do i test the changes we made in task 2
```

ChatGPT

Avi, your game looks correct! The **Possible range: 1 - 100** is visible. Now let's test whether it updates properly after each guess.

Step 1 — Test a low guess

In the game window, click the white input box, type 25, and click **SUBMIT**.

Expected result:

- Feedback says T00 LOW! if the secret number is greater than 25.
- If the secret number is greater than 25, the range minimum should become 26.
- Attempts should increase to 1.

If the secret number is below 25, the guess will be too high instead. That's okay—we'll test based on the feedback.

Step 2 — Test the updated range

Look at the feedback message and follow the matching case:

Feedback Expected range
--- ---
T00 LOW! after guessing 25 26 - 100
T00 HIGH! after guessing 25 1 - 24

The range should exclude numbers you've already ruled out.

Step 3 — Test another guess

Use the updated range to make another guess.

For example, if your range became 26 - 100, enter 60:

- If it's too low, the range should become 61 - 100.
- If it's too high, the range should become 26 - 59.
- If it's correct, you win.

Step 4 — Test the empty-input fix again

Clear the input box and click **SUBMIT**.

Expected: the warning appears, the game stays open, and the attempt count does not increase.

Step 5 — Tell me what happened

First, enter `25` and click SUBMIT. Send me a screenshot showing the updated range and feedback. I'll confirm whether Task 2 is working before we commit it.

User

its working whats next

ChatGPT

Great, Avi! ☐ Task 2 is working. Now we'll commit it separately, then move on to **Task 3: Recent Guess History Tracker**.

Step 1 — Commit Task 2

Stop the game window if it's still running, then go to your VS Code terminal.

Run these commands one at a time:

1. Stage the updated file

```
git add game/game_engine.py
```

2. Commit Task 2

```
git commit -m "Implement dynamic search range"
```

3. Push to your GitHub repository

```
git push origin main
```

You should now have separate commits for the bug fix and the dynamic range feature.

Step 2 — Add the guess history feature

We'll display previous guesses on the game screen with color-coded feedback.

For example:

```
<box border radius="lg" padding={3} gap={2}>
<title size="lg">Recent Guess History</title>
<row align=center>
<box background="#FCE8E6" radius="md" padding={2} width="64px" align=center>
<text color="#B3261E" weight="medium">25</text>
</box>
<box flex="1">
<text color="#B3261E" weight="medium">TOO LOW</text>
</box>
</row>
<row align=center>
<box background="#FCE8E6" radius="md" padding={2} width="64px" align=center>
<text color="#B3261E" weight="medium">60</text>
</box>
<box flex="1">
<text color="#B3261E" weight="medium">TOO LOW</text>
</box>
</row>
```

```

<row align=center>
<box background="#DDF5E1" radius="md" padding={2} width="64px" align=center>
<text color="#166534" weight="medium">83</text>
</box>
<box flex="1">
<text color="#166534" weight="medium">TOO HIGH</text>
</box>
</row>
<text color="secondary" size="xs">Illustration only — your actual guesses and colors will
depend on the game.</text>
</box>

```

We'll keep the latest several valid guesses, show whether each was too high or too low, and clear the history when a new game starts.

Step 3 — Make one small change first

Open `game/game_engine.py`.

Inside `__init__()`, find:

```
self.max_range = 100
```

Immediately below it, add:

```
self.guess_history = []
```

This creates an empty list to store guesses and their results.

Don't change anything else yet. Tell me once you've committed Task 2 and added this line. Then I'll give you the exact next code changes for recording guesses and displaying the history panel.

User

give me the updated game engine py file

ChatGPT

Avi, here's the complete updated `game_engine.py` with **Task 1 (empty input crash fix)**, **Task 2 (dynamic search range)**, and **Task 3 (recent guess history)** implemented.

Replace all the code in `game/game_engine.py` with the following.

```

<CodeBlock language="python" editable>
import random
import pygame
from game.text_box import TextBox

class GameEngine:
def init(self, width, height):
self.width = width
self.height = height
self.secret_number = random.randint(1, 100)
self.attempts = 0

```

```

# Task 2: Dynamic search range
self.min_range = 1
self.max_range = 100

# Task 3: Recent guess history
self.guess_history = []
self.max_history = 5

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False

self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

self.font_title = pygame.font.SysFont(None, 42)
self.font_medium = pygame.font.SysFont(None, 28)
self.font_btn = pygame.font.SysFont(None, 26)
self.font_small = pygame.font.SysFont(None, 24)

def submit_guess(self):
    if self.game_won:
        return

    # Task 1: Prevent empty input from crashing the game
    if not self.input_box.text.strip():
        self.feedback_msg = "Please enter a number!"
        self.feedback_color = (255, 220, 80)
        return

    # Accept only integer input
    try:
        guess = int(self.input_box.text)
    except ValueError:
        self.feedback_msg = "Please enter a valid number!"
        self.feedback_color = (255, 220, 80)
        return

    # Keep guesses within the allowed range
    if not 1 <= guess <= 100:
        self.feedback_msg = "Enter a number between 1 and 100!"
        self.feedback_color = (255, 220, 80)
        return

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.minrange = max(self.minrange, guess + 1)
        result = "TOO LOW"

```

```

self.feedback_msg = f"TOO LOW! (Guess was {guess})"
self.feedback_color = (80, 160, 240)

elif guess > self.secret_number:
self.maxrange = min(self.maxrange, guess - 1)
result = "TOO HIGH"
self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
self.feedback_color = (240, 100, 80)

else:
result = "CORRECT"
self.feedback_msg = (
f"CORRECT! Found in {self.attempts} attempts."
)
self.feedback_color = (80, 220, 90)
self.game_won = True

# Store the guess and its result
self.guess_history.append((guess, result))

# Keep only the latest guesses
self.guesshistory = self.guesshistory[-self.max_history:]

def reset(self):
self.secret_number = random.randint(1, 100)
self.attempts = 0
self.min_range = 1
self.max_range = 100

# Clear history for the new game
self.guess_history.clear()

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False
self.input_box.clear()

def handle_event(self, event):
self.inputbox.handleevent(event)

if event.type == pygame.KEYDOWN:
if event.key == pygame.K_RETURN:
self.submit_guess()
elif event.key == pygame.Kr and self.gamewon:
self.reset()

elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
if self.submit_btn.collidepoint(event.pos):
self.submit_guess()

def update(self):

```



```

pass

def render(self, screen):
    screen.fill((30, 34, 42))

    titlesurf = self.fonttitle.render(
        "Number Guessing Arena", True, (245, 245, 245)
    )
    screen.blit(
        title_surf,
        (self.width // 2 - titlesurf.getwidth() // 2, 35),
    )

    attemptssurf = self.fontmedium.render(
        f"Attempts: {self.attempts}", True, (180, 185, 195)
    )
    screen.blit(
        attempts_surf,
        (self.width // 2 - attemptssurf.getwidth() // 2, 95),
    )

    # Task 2: Display the possible search range
    rangesurf = self.fontmedium.render(
        f"Possible range: {self.minrange} - {self.maxrange}",
        True,
        (180, 185, 195),
    )
    screen.blit(
        range_surf,
        (self.width // 2 - rangesurf.getwidth() // 2, 120),
    )

    self.input_box.render(screen)

    pygame.draw.rect(
        screen, (50, 150, 80), self.submitbtn, borderradius=6
    )
    pygame.draw.rect(
        screen,
        (220, 220, 220),
        self.submit_btn,
        width=2,
        border_radius=6,
    )

    btntext = self.fontbtn.render(
        "SUBMIT", True, (255, 255, 255)
    )

```

```

screen.blit(
    btn_text,
    (
        self.submitbtn.centerx - btntext.get_width() // 2,
        self.submitbtn.centery - btntext.get_height() // 2,
    ),
)

feedbacksurf = self.fontmedium.render(
    self.feedbackmsg, True, self.feedbackcolor
)

screen.blit(
    feedback_surf,
    (
        self.width // 2 - feedbacksurf.getwidth() // 2,
        235,
    ),
)

# Task 3: Draw recent guess history
historytitle = self.fontmedium.render(
    "Recent Guess History", True, (245, 245, 245)
)

screen.blit(
    history_title,
    (self.width // 2 - historytitle.getwidth() // 2, 300),
)

history_y = 335

for guess, result in reversed(self.guess_history):
    if result == "TOO LOW":
        color = (80, 160, 240)
    elif result == "TOO HIGH":
        color = (240, 100, 80)
    else:
        color = (80, 220, 90)

    historytext = self.fontsmall.render(
        f"{guess} - {result}", True, color
    )

    screen.blit(
        history_text,
        (
            self.width // 2 - historytext.getwidth() // 2,
            history_y,
        ),
    ),

```

```

)
history_y += 27
if self.game_won:
    restartsurf = self.fontmedium.render(
    "Press [R] to Start a New Game",
    True,
    (255, 220, 80),
    )
    screen.blit(
    restart_surf,
    (
    self.width // 2 - restartsurf.getwidth() // 2,
    max(500, history_y + 15),
    ),
    )
</CodeBlock>

```

What to do now

1. Open game/game_engine.py in VS Code.
2. Replace its contents with the code above.
3. Press Ctrl + S.
4. Run the game:

```
python main.py
```

Test these four things

- **Empty input:** the game displays a warning without crashing.
- **Dynamic range:** low and high guesses update the possible range.
- **Guess history:** previous valid guesses appear with blue, red, or green feedback.
- **Restart:** after winning, press R and check that the range returns to 1 - 100 and the history clears.

The history panel displays the five most recent valid guesses. Invalid or empty inputs do not count as attempts or appear in the history.

Once everything works, tell me. We'll commit Task 3 separately, then implement Task 4: the maximum attempt limit and Game Over screen.

User

everything is working but the window is too small i cant see all history i can see only top 2 recent ones and the text press r to reset also dissapears

ChatGPT

Avi, the issue is that your game is drawing the history below the available window space. We'll fix this by **making the game window taller and displaying the restart message in a fixed position** so it doesn't disappear.

Let's do it step by step.

Step 1 — Increase the game window height

Open `main.py` in VS Code.

Look for where the window dimensions are defined. It will likely look something like:

```
WIDTH = 800
HEIGHT = 600
```

Change it to:

```
WIDTH = 900
HEIGHT = 800
```

If your variables have different names, keep those names and change only the values.
Save with `Ctrl + S`.

Step 2 — Fix the history and restart message

Open `game/game_engine.py`.

Inside `render()`, find this line:

```
history_y = 335
```

Replace the entire section starting from `history_y = 335` through the `if self.game_won:` block at the end of `render()` with this:

<CodeBlock language="python" editable>

```
history_y = 335
```

```
line_height = 27
```

```
# Show the most recent guesses that fit in the window
```

```
available_height = self.height - 420
```

```
visiblecount = max(0, availableheight // line_height)
```

```
visiblehistory = self.guesshistory[-visiblecount:] if visiblecount else []
```

```
for guess, result in reversed(visible_history):
```

```
if result == "TOO LOW":
```

```
color = (80, 160, 240)
```

```
elif result == "TOO HIGH":
```

```
color = (240, 100, 80)
```

```
else:
```

```
color = (80, 220, 90)
```

```
historytext = self.fontsmall.render(
```

```
f"{guess} - {result}", True, color
```

```
)
```

```
screen.blit(
```

```
history_text,
```

```
(
```

```
self.width // 2 - historytext.getwidth() // 2,
```

```
history_y,
```

```
),
```

```

)
historyy += lineheight
# Keep the restart instruction in a fixed visible position
if self.game_won:
    restartsurf = self.fontmedium.render(
        "Press [R] to Start a New Game",
        True,
        (255, 220, 80),
    )
    restart_y = self.height - 50
    screen.blit(
        restart_surf,
        (
            self.width // 2 - restartsurf.getwidth() // 2,
            restart_y,
        ),
    )
</CodeBlock>

```

Save the file.

Step 3 — Run and test

In the VS Code terminal, run:

```
python main.py
```

Check that:

- The window is taller.
- The recent guesses are visible.
- The restart message stays near the bottom after winning.
- Pressing R starts a fresh game and clears the history.

One important note: this displays only the history entries that fit in the window, up to the five guesses currently stored. If you want all five visible comfortably, the taller window should give you more room.

Tell me how it looks after these changes, and then we'll commit Task 3 before implementing the maximum-attempt limit.

User

give me the entire changed game engine py file

ChatGPT

Avi, replace the entire contents of game/game_engine.py with the code below. This includes **Tasks 1, 2, and 3**, with the history layout adjusted to fit a taller window and the restart message kept in a fixed position near the bottom.

Important: This file assumes you've already changed the window dimensions in main.py to 900 × 800, as discussed.

```

<CodeBlock language="python" editable>
import random
import pygame
from game.text_box import TextBox

class GameEngine:
def init(self, width, height):
self.width = width
self.height = height
self.secret_number = random.randint(1, 100)
self.attempts = 0

# Task 2: Dynamic search range
self.min_range = 1
self.max_range = 100

# Task 3: Recent guess history
self.guess_history = []
self.max_history = 5

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False

self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

self.font_title = pygame.font.SysFont(None, 42)
self.font_medium = pygame.font.SysFont(None, 28)
self.font_btn = pygame.font.SysFont(None, 26)
self.font_small = pygame.font.SysFont(None, 24)

def submit_guess(self):
if self.game_won:
return

# Task 1: Handle empty input safely
if not self.input_box.text.strip():
self.feedback_msg = "Please enter a number!"
self.feedback_color = (255, 220, 80)
return

try:
guess = int(self.input_box.text)
except ValueError:
self.feedback_msg = "Please enter a valid number!"
self.feedback_color = (255, 220, 80)
return

# Validate guesses are within the game's allowed range

```

```

if not 1 <= guess <= 100:
self.feedback_msg = "Enter a number between 1 and 100!"
self.feedback_color = (255, 220, 80)
return

self.attempts += 1
self.input_box.clear()

if guess < self.secret_number:
self.minrange = max(self.minrange, guess + 1)
result = "TOO LOW"
self.feedback_msg = f"TOO LOW! (Guess was {guess})"
self.feedback_color = (80, 160, 240)

elif guess > self.secret_number:
self.maxrange = min(self.maxrange, guess - 1)
result = "TOO HIGH"
self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
self.feedback_color = (240, 100, 80)

else:
result = "CORRECT"
self.feedback_msg = (
f"CORRECT! Found in {self.attempts} attempts."
)
self.feedback_color = (80, 220, 90)
self.game_won = True

# Store valid guesses and keep only the most recent five
self.guess_history.append((guess, result))
self.guesshistory = self.guesshistory[-self.max_history:]

def reset(self):
self.secret_number = random.randint(1, 100)
self.attempts = 0
self.min_range = 1
self.max_range = 100
self.guess_history.clear()

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False
self.input_box.clear()

def handle_event(self, event):
self.inputbox.handleevent(event)

if event.type == pygame.KEYDOWN:
if event.key == pygame.K_RETURN:
self.submit_guess()

```

```

elif event.key == pygame.Kr and self.gamewon:
self.reset()

elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
if self.submit_btn.collidepoint(event.pos):
self.submit_guess()

def update(self):
pass

def render(self, screen):
screen.fill((30, 34, 42))

# Game title
titlesurf = self.fonttitle.render(
"Number Guessing Arena", True, (245, 245, 245)
)
screen.blit(
title_surf,
(self.width // 2 - titlesurf.getwidth() // 2, 35),
)

# Attempts counter
attemptssurf = self.fontmedium.render(
f"Attempts: {self.attempts}", True, (180, 185, 195)
)
screen.blit(
attempts_surf,
(self.width // 2 - attemptssurf.getwidth() // 2, 95),
)

# Task 2: Display possible search range
rangesurf = self.fontmedium.render(
f"Possible range: {self.minrange} - {self.maxrange}",
True,
(180, 185, 195),
)
screen.blit(
range_surf,
(self.width // 2 - rangesurf.getwidth() // 2, 120),
)

# Input box and submit button
self.input_box.render(screen)

pygame.draw.rect(
screen, (50, 150, 80), self.submitbtn, borderradius=6
)
pygame.draw.rect(

```



```

screen, (220, 220, 220), self.submit_btn,
width=2, border_radius=6
)
btn_text = self.fontbtn.render(
"SUBMIT", True, (255, 255, 255)
)
screen.blit(
btn_text,
(
self.submitbtn.centerx - btn_text.get_width() // 2,
self.submitbtn.centery - btn_text.get_height() // 2,
),
)
# Feedback message
feedbacksurf = self.fontmedium.render(
self.feedbackmsg, True, self.feedbackcolor
)
screen.blit(
feedback_surf,
(
self.width // 2 - feedbacksurf.getwidth() // 2,
235,
),
)
# Task 3: Recent guess history
historytitle = self.fontmedium.render(
"Recent Guess History", True, (245, 245, 245)
)
historytitley = 290
screen.blit(
history_title,
(
self.width // 2 - historytitle.getwidth() // 2,
historytitley,
),
)
line_height = 30
historystarty = historytitley + 35
# Reserve space at the bottom for the restart instruction
bottommargin = 65 if self.gamewon else 20
available_height = max(
0, self.height - historystarty - bottom_margin

```

```

)
visible_count = min(
self.max_history,
availableheight // lineheight,
len(self.guess_history),
)

visiblehistory = self.guessshistory[-visiblecount:] if visiblecount else []

for guess, result in reversed(visible_history):
if result == "TOO LOW":
color = (80, 160, 240)
elif result == "TOO HIGH":
color = (240, 100, 80)
else:
color = (80, 220, 90)

historytext = self.fontsmall.render(
f"{guess} - {result}", True, color
)
screen.blit(
history_text,
(
self.width // 2 - historytext.getwidth() // 2,
historystarty,
),
)
historystarty += line_height

# Keep the restart message at a fixed position near the bottom
if self.game_won:
restartsurf = self.fontmedium.render(
"Press [R] to Start a New Game",
True,
(255, 220, 80),
)
restart_y = self.height - 45
screen.blit(
restart_surf,
(
self.width // 2 - restartsurf.getwidth() // 2,
restart_y,
),
)

```

</CodeBlock>

Now do this

1. Save with Ctrl + S.
2. Confirm that main.py uses a window size of 900 × 800.
3. Run python main.py.
4. Make several guesses and check that the history is visible.
5. Win the game and confirm that **Press [R] to Start a New Game** remains visible.
6. Press R and verify that the history and range reset.

Once you've verified it, we'll commit Task 3 separately and implement Task 4: the maximum-attempt limit and Game Over screen.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file
000000003e8c82109c5f1a4689bbca9a?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":102368,"width":1352,"height":922,"metadata":{"sanitized":true}}
```

its like this

ChatGPT

Avi, I can see the issue! Your game is running, but **only one history entry is visible**, and the window is still using a relatively small height. The range feature is working correctly (61–100 after guessing 60).

Let's fix the layout before moving to Task 4.

Step 1 — Maximize the game window

Click the square button next to the minimize button in the game's title bar. This will maximize the window and give the history more room.

If you still can't see all five guesses, do Step 2 as well.

Step 2 — Adjust the history layout

In game/game_engine.py, find this section inside render():

```
history_title_y = 290
```

Replace the history section starting at # Task 3: Recent guess history and ending just before # Keep the restart message at a fixed position near the bottom with this code:

```
<CodeBlock language="python" editable>
# Task 3: Recent guess history
historytitle = self.fontmedium.render(
    "Recent Guess History", True, (245, 245, 245)
)
historytitle_y = 285
screen.blit(
    history_title,
    (
        self.width // 2 - historytitle.getwidth() // 2,
```

```

historytitley,
),
)
line_height = 30
historystarty = historytitley + 35
# Show the most recent five guesses
for guess, result in reversed(self.guess_history[-5:]):
if result == "TOO LOW":
color = (80, 160, 240)
elif result == "TOO HIGH":
color = (240, 100, 80)
else:
color = (80, 220, 90)
historytext = self.fontsmall.render(
f"{guess} - {result}", True, color
)
screen.blit(
history_text,
(
self.width // 2 - historytext.getwidth() // 2,
historystarty,
),
)
historystarty += line_height
</CodeBlock>

```

This removes the calculation that hides history entries when the available-height calculation is too restrictive. The code now draws every stored entry, up to five.

Step 3 — Keep the restart message visible

Make sure the end of `render()` still contains this block:

```

if self.game_won:
    restart_surf = self.font_medium.render(
        "Press [R] to Start a New Game",
        True,
        (255, 220, 80),
    )
    restart_y = self.height - 45
    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            restart_y,
        ),
    )

```

This keeps the restart instruction near the bottom of the game window after a win.

Step 4 — Test again

1. Save with Ctrl + S.
2. Close the running game.
3. Run `python main.py` again.
4. Make several guesses and check the history panel.

One detail: your code stores only the **five most recent guesses**, so you should see up to five history entries. The restart message appears only after you guess the secret number correctly.

Send me another screenshot after testing. Once the layout is good, we'll commit Task 3 and finish the maximum-attempt feature.

User

the square is not clickable

ChatGPT

No problem, Avi. The maximize button might not be responding because the game window is partially covered or Windows isn't handling the click.

Let's resize it another way.

Step 1 — Close the game

Click the **X** at the top-right of the game window. Don't worry; your code changes are saved in VS Code.

Step 2 — Change the window size in `main.py`

1. Open `main.py` in VS Code.
2. Look for the line that creates the Pygame window. It should look similar to:

```
screen = pygame.display.set_mode((width, height))
```

Or you may see something like:

```
WIDTH = 800  
HEIGHT = 600
```

3. Send me the part of `main.py` where the window size is defined and `pygame.display.set_mode()` is called.

Don't change it yet. I'll tell you exactly which lines to update so the game window is tall enough to show all five history entries and the restart message without overlapping anything.

User

does it matter or am i doing extra stuff

ChatGPT

Avi, **you're doing a little extra work here.** The maximize button and window resizing are not essential to your lab submission.

Your lab requires you to fix the bug, implement the three features, record before-and-after videos, and provide your chat history.

What matters vs. what doesn't

Item	Priority
Empty input no longer crashes	Required ☐
Dynamic search range works	Required ☐
Recent guess history works	Required ☐
Maximum attempts and Game Over screen	Required — still to do
Before and after 10-second videos	Required
Separate Git commits for each task	Required
Making the window bigger or maximizing it	Optional

What I recommend

1. **Stop adjusting the window size for now.** Your current game already runs and displays the range and history.
2. Commit Task 2 if you haven't already, then finish and commit Task 3.
3. Implement Task 4, the maximum-attempt limit and Game Over screen.
4. Test the required features, record your after video, and prepare the submission.

One thing to keep in mind: your history currently stores up to five guesses, but you should verify the history feature works as required. It doesn't have to look perfect; it just needs to be usable and visible enough to demonstrate.

Let's move forward. Have you already committed and pushed Task 3, or is it still uncommitted?

User

okay lets not do all this lets do whatever is required , give me the required game engine py code

ChatGPT

Avi, let's keep it simple. This version includes **all four required tasks** from your lab README: the empty-input fix, dynamic search range, recent guess history, and maximum attempts with a Game Over screen. I've kept the existing design rather than spending more time resizing the window.

Replace the entire contents of `game/game_engine.py` with this code.

```
<CodeBlock language="python" editable>
```

```
import random
import pygame
from game.text_box import TextBox
```

```
class GameEngine:
    def init(self, width, height):
        self.width = width
        self.height = height
```

```

self.max_attempts = 10
self.max_history = 5
self.secret_number = random.randint(1, 100)
self.attempts = 0
self.min_range = 1
self.max_range = 100
self.guess_history = []

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False
self.game_over = False

self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

self.font_title = pygame.font.SysFont(None, 42)
self.font_medium = pygame.font.SysFont(None, 28)
self.font_btn = pygame.font.SysFont(None, 26)
self.font_small = pygame.font.SysFont(None, 24)

def submit_guess(self):
    if self.gamewon or self.gameover:
        return

    # Task 1: Handle empty input without crashing
    text = self.input_box.text.strip()

    if not text:
        self.feedback_msg = "Please enter a number!"
        self.feedback_color = (255, 220, 80)
        return

    try:
        guess = int(text)
    except ValueError:
        self.feedback_msg = "Please enter a valid number!"
        self.feedback_color = (255, 220, 80)
        return

    if not 1 <= guess <= 100:
        self.feedback_msg = "Enter a number between 1 and 100!"
        self.feedback_color = (255, 220, 80)
        return

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        result = "TOO LOW"

```

```

self.minrange = max(self.minrange, guess + 1)
self.feedback_msg = f"TOO LOW! (Guess was {guess})"
self.feedback_color = (80, 160, 240)

elif guess > self.secret_number:
    result = "TOO HIGH"
    self.maxrange = min(self.maxrange, guess - 1)
    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)

else:
    result = "CORRECT"
    self.feedback_msg = (
        f"CORRECT! Found in {self.attempts} attempts."
    )
    self.feedback_color = (80, 220, 90)
    self.game_won = True

# Task 3: Store the latest valid guesses
self.guess_history.append((guess, result))
self.guesshistory = self.guesshistory[-self.max_history:]

# Task 4: End the game after the attempt limit
if not self.gamewon and self.attempts >= self.maxattempts:
    self.game_over = True
    self.feedback_msg = (
        f"GAME OVER! The number was {self.secret_number}."
    )
    self.feedback_color = (255, 100, 100)

def reset(self):
    self.secret_number = random.randint(1, 100)
    self.attempts = 0
    self.min_range = 1
    self.max_range = 100
    self.guess_history.clear()

    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.game_over = False
    self.input_box.clear()

def handle_event(self, event):
    self.inputbox.handleevent(event)

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_r and (
            self.gamewon or self.gameover

```



```

):
self.reset()
elif event.key == pygame.K_RETURN:
self.submit_guess()

elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
if self.submit_btn.collidepoint(event.pos):
self.submit_guess()

def update(self):
pass

def render(self, screen):
screen.fill((30, 34, 42))

titlesurf = self.fonttitle.render(
"Number Guessing Arena", True, (245, 245, 245)
)
screen.blit(
title_surf,
(self.width // 2 - titlesurf.getwidth() // 2, 35),
)

attemptssurf = self.fontmedium.render(
f"Attempts: {self.attempts}/{self.max_attempts}",
True,
(180, 185, 195),
)
screen.blit(
attempts_surf,
(self.width // 2 - attemptssurf.getwidth() // 2, 95),
)

# Task 2: Display the narrowing search range
rangesurf = self.fontmedium.render(
f"Possible range: {self.minrange} - {self.maxrange}",
True,
(180, 185, 195),
)
screen.blit(
range_surf,
(self.width // 2 - rangesurf.getwidth() // 2, 120),
)

self.input_box.render(screen)

pygame.draw.rect(
screen, (50, 150, 80), self.submitbtn, borderradius=6
)

```

```

pygame.draw.rect(
screen, (220, 220, 220), self.submit_btn,
width=2, border_radius=6
)

btntext = self.fontbtn.render(
"SUBMIT", True, (255, 255, 255)
)
screen.blit(
btn_text,
(
self.submitbtn.centerx - btntext.get_width() // 2,
self.submitbtn.centery - btntext.get_height() // 2,
),
)

feedbacksurf = self.fontmedium.render(
self.feedbackmsg, True, self.feedbackcolor
)
screen.blit(
feedback_surf,
(
self.width // 2 - feedbacksurf.getwidth() // 2,
235,
),
)

# Task 3: Recent guess history
historytitle = self.fontmedium.render(
"Recent Guess History", True, (245, 245, 245)
)
screen.blit(
history_title,
(
self.width // 2 - historytitle.getwidth() // 2,
290,
),
)

history_y = 330
for guess, result in reversed(self.guess_history):
if result == "TOO LOW":
color = (80, 160, 240)
elif result == "TOO HIGH":
color = (240, 100, 80)
else:

```

```

color = (80, 220, 90)
historysurf = self.fontsmall.render(
f"{guess} - {result}", True, color
)
screen.blit(
history_surf,
(
self.width // 2 - historysurf.getwidth() // 2,
history_y,
),
)
history_y += 27

# Task 4: Win or failure state
if self.game_won:
message = "You won! Press [R] to play again"
elif self.game_over:
message = "Game over! Press [R] to try again"
else:
message = None

if message:
restartsurf = self.fontmedium.render(
message, True, (255, 220, 80)
)
restart_y = self.height - 40
screen.blit(
restart_surf,
(
self.width // 2 - restartsurf.getwidth() // 2,
restart_y,
),
)

```

Now do just these steps

1. Replace the code and press Ctrl + S.
2. Run `python main.py`.
3. Test an empty submission, a low/high guess, and the updated range.
4. Check that recent guesses appear.
5. Make 10 valid incorrect guesses to test Game Over, or guess correctly to test winning.
6. After either ending, press R to confirm the game resets.

The attempt limit is set to **10 valid guesses**. Empty or invalid submissions don't consume attempts. A correct guess on the tenth attempt still wins.

Once you've confirmed it works, we'll create the separate Task 3 and Task 4 commits, then finish the GitHub push and required submission items.

User

okay its working whats next

ChatGPT

Great, Avi! ☐ All four tasks are now implemented. Let's finish the lab submission properly.

Step 1 — Check your Git changes

Stop the running game, open the VS Code terminal, and run:

```
git status
```

You should see `game/game_engine.py` listed as modified.

Step 2 — Create separate commits for Tasks 3 and 4

Your lab requires separate commits for each task. Since the current file contains both new features, we'll use Git's interactive staging to keep the commits separate.

First, save a backup of the working file:

```
Copy-Item game/game_engine.py game_engine_backup.py
```

Now, the easiest safe approach is to create **two commits using separate versions of the file**. Because the current file includes both Tasks 3 and 4, don't simply commit the entire file twice—that wouldn't create meaningful separate commits.

Instead, since Tasks 1 and 2 are already committed, you can use these commands to save your completed implementation in a single commit if your instructor accepts it:

```
git add game/game_engine.py
git commit -m "Add guess history and maximum attempts"
git push origin main
```

Important: This groups Tasks 3 and 4 into one commit, so it does not fully satisfy the separate-commit requirement. If your instructor strictly checks commit history, tell me and I'll guide you through separating the changes correctly.

Step 3 — Record your AFTER video

Record a 10-second clip showing the completed game. Try to demonstrate:

- The possible search range.
- The recent guess history.
- The attempt counter and Game Over screen, if possible.

Use Win + Alt + R to record if Xbox Game Bar is working.

Step 4 — Prepare your submission

Your README specifies these three deliverables:

- [] 10-second video of the original game showing the bug.
- [] 10-second video of the completed game.
- [] Link to the AI chat page with the complete conversation history.

Upload these to your lab repository under the Lab-4 folder, following your instructor's submission setup.

Step 5 — Verify GitHub

Open your personal repository:

[Open your 60numbreguess repository](https://github.com/Avi2106/60numbreguess)

Check that the latest `game/game_engine.py` contains the completed code.

Let's do Step 1 first: Run `git status` and send me the output. I'll help you choose the correct commit approach based on the changes Git detects.

User

Changes not staged for commit:

(use "`git add <file>...`" to update what will be committed)

(use "`git restore <file>...`" to discard changes in working directory)

modified: `game/game_engine.py`

Untracked files:

(use "`git add <file>...`" to include in what will be committed)

`game/pycache/`

no changes added to commit (use "`git add`" and/or "`git commit -a`")

`git status`

ChatGPT

Perfect, Avi. Your output is normal. `game/game_engine.py` has your latest changes, and `game/__pycache__` contains Python-generated cache files that we don't need to submit.

Let's clean that up and commit your work.

Step 1 — Ignore the cache files

In your VS Code terminal, run:

```
Add-Content .gitignore "`n__pycache__/"
```

This prevents Python cache folders from being tracked by Git.

Step 2 — Commit your completed features

Your current file contains both Task 3 and Task 4. Since they are already implemented together, run:

```
git add game/game_engine.py .gitignore
git commit -m "Add guess history and maximum attempts"
```

Then push to GitHub:

```
git push origin main
```

Note: This creates one commit for Tasks 3 and 4 together. Your lab asks for separate commits for each task, so if your instructor checks commit history strictly, we'll need to separate those changes into two commits instead.

Step 3 — Verify the push

Run:

```
git status
```

Ideally, you'll see:

```
nothing to commit, working tree clean
```

Step 4 — Finish your deliverables

- [] Before video: original game showing the empty-input crash.
- [] After video: updated game showing the range, guess history, and maximum-attempt Game Over screen.
- [] Chat history: share the link to this conversation with the complete history, as your README requests.
- [] Updated code pushed to your personal GitHub repository.

Do Steps 1-3 now and send me the output of `git status`. Then we'll verify your repository and make sure nothing required is missing.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file  
0000000089888210b1dae66f139f069d?sharedconversation  
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size  
bytes":58601,"width":1326,"height":302,"metadata":{"sanitized":true}}
```

ChatGPT

You're doing it correctly, Avi! ☐ The screenshot shows that both files have been staged successfully. The LF will be replaced by CRLF message is just a Windows line-ending warning, not an error.

Step 1 — Commit your changes

Copy and run:

```
git commit -m "Add guess history and maximum attempts"
```

Step 2 — Push to GitHub

After the commit succeeds, run:

```
git push origin main
```

Step 3 — Verify everything

Run:

```
git status
```

You want to see:

```
nothing to commit, working tree clean
```

Then open your [GitHub repository](<https://github.com/Avi2106/60numberguess>) and check that the latest code is there.

One thing to remember: Tasks 3 and 4 are currently combined in one commit. Your lab asks for separate commits per task, so this may need to be corrected if your instructor checks the commit history strictly.

Run Step 1 now and send me the output.

User

```
{"contentType":"imageassetpointer","assetpointer":"sediment://file
00000000416082108ed67e5bb0cae308?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":124038,"width":1270,"height":710,"metadata":{"sanitized":true}}
```

ChatGPT

Perfect, Avi! ☐ Your code has been pushed successfully to your personal GitHub repository, and `git status` confirms that your working tree is clean.

Your repository: [Avi2106/60numberguess](https://github.com/Avi2106/60numberguess)

What's left to finish the lab?

```
{@body const tasks = [{label:"Fix empty input crash",detail:"Task 1 — committed and
pushed"},{label:"Dynamic search range",detail:"Task 2 — committed and
pushed"},{label:"Recent guess history",detail:"Task 3 — implemented and
pushed"},{label:"Maximum attempts and Game Over",detail:"Task 4 — implemented and
pushed"},{label:"Record before video",detail:"10 seconds showing the original
empty-input crash"},{label:"Record after video",detail:"10 seconds showing the new
features working"},{label:"Prepare chat history link",detail:"Share this conversation with
the complete history"}]}
```

```
{@body const [done,setDone] = DIL.useState([0,1,2,3])}
<box border radius="lg" padding={3} gap={3}>
  {#each tasks as task,i}
    <checkbox key={task.label} checked={done.includes(i)}
      onChange={v=>setDone(old=>v?[...old,i]:old.filter(n=>n!==i))} lineThrough={false}>
      {task.label}
    <text color="secondary" size="sm">{task.detail}</text>
  </checkbox>
</each>
</box>
```

Important: one thing to check

Your lab specifically asks for **separate commits for each task**. Tasks 3 and 4 were committed together, so the code is complete, but your commit history may not strictly satisfy that requirement.

If your instructor checks individual commits, we should separate them before finishing.

Next step

Have you already recorded both the 10-second before video and the 10-second after video?

Tell me yes or no, and I'll guide you through the remaining submission steps.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file
00000000b99c82108e3ee70550bd7e4e?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":249111,"width":2047,"height":1199,"metadata":{"sanitized":true}}
```

ChatGPT

Avi, this screenshot confirms that your code is pushed to GitHub and you have **3 commits**:

1. Fix empty input crash ☐
2. Implement dynamic search range ☐
3. Add guess history and maximum attempts ☐

Your code is uploaded, but Tasks 3 and 4 are combined into one commit. Since the lab asks for separate commits for each task, that is the only Git requirement that may still need attention.

What you need to finish now

- [] Record the **10-second BEFORE video** showing the original empty-input crash.
- [] Record the **10-second AFTER video** showing the completed game, including range, history, and Game Over.
- [] Save the link to this ChatGPT conversation with the full chat history.
- [] Upload these deliverables to the Lab-4 folder in your lab repository.

One question: Have you recorded the before and after videos yet? If not, I'll guide you through recording them on Windows, step by step.

User

we need separate commits for 3 and 4 how do i fix this now

ChatGPT

Avi, we can fix this without losing your working code or starting the lab again. We'll keep your existing Task 1 and Task 2 commits and split the combined Task 3 and Task 4 commit into two separate commits.

Your current history is:

1. Fix empty input crash
2. Implement dynamic search range
3. Add guess history and maximum attempts — we'll split this into two.

We'll first make a backup, then separate the features, and finally push the corrected history to GitHub.

Avi, we can fix this without losing your working code or starting the lab again. We'll keep your existing Task 1 and Task 2 commits and split the combined Task 3 and Task 4 commit into two separate commits.

Your current history is:

1. Fix empty input crash

2. Implement dynamic search range
3. Add guess history and maximum attempts — we'll split this into two.

We'll first make a backup, then separate the features, and finally push the corrected history to GitHub.

Step 1 — Back up your completed code

In your VS Code PowerShell terminal, run:

```
Copy-Item game/game_engine.py "$HOME\game_engine_final_backup.py"
```

This saves your completed version outside the project folder, so we can restore it later.

Step 2 — Undo only the combined commit, keeping your code

Run:

```
git reset --mixed HEAD~1
```

Don't worry: this does **not** delete your code. It removes the latest commit from your local history while keeping the changes in your working directory.

Your two earlier commits will remain intact.

Step 3 — Confirm the state

Run:

```
git log -3 --oneline  
git status
```

You should see the two earlier commits, and your completed code should appear as modified or untracked changes.

Stop here and send me a screenshot of the output. I'll guide you through creating a genuine Task 3-only commit first, then restoring the backed-up code and creating a separate Task 4 commit. After that, we'll push the corrected history to GitHub.

Don't run `git push` yet; we'll push after both commits are correct.

Step 1 — Back up your completed code

In your VS Code PowerShell terminal, run:

```
Copy-Item game/game_engine.py "$HOME\game_engine_final_backup.py"
```

This saves your completed version outside the project folder, so we can restore it later.

Step 2 — Undo only the combined commit, keeping your code

Run:

```
git reset --mixed HEAD~1
```

Don't worry: this does **not** delete your code. It removes the latest commit from your local history while keeping the changes in your working directory.

Your two earlier commits will remain intact.

Step 3 — Confirm the state

Run:

```
git log -3 --oneline  
git status
```

You should see the two earlier commits, and your completed code should appear as modified or untracked changes.

Stop here and send me a screenshot of the output. I'll guide you through creating a genuine Task 3-only commit first, then restoring the backed-up code and creating a separate Task 4 commit. After that, we'll push the corrected history to GitHub.

Don't run `git push` yet; we'll push after both commits are correct.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file  
000000004a8c82109b6b9e28e1f5862c?sharedconversation  
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size  
bytes":141534,"width":1450,"height":938,"metadata":{"sanitized":true}}
```

ChatGPT

Good, Avi. ☐ The reset worked, and your code changes are still present.

Your screenshot confirms:

- Task 1 and Task 2 commits are intact.
- `game/game_engine.py` contains uncommitted changes.
- Your local branch is one commit behind GitHub because the combined commit still exists on the remote.

We'll now create a proper Task 3 commit followed by a Task 4 commit.

Step 1 — Restore the original Task 3-only version

First, run this command to check that your backup exists:

```
Test-Path "$HOME\game_engine_final_backup.py"
```

It should return `True`.

Don't run anything else yet. Send me the result. We'll use that backup to preserve the complete version while separating the commits safely.

User

it gave true

ChatGPT

Perfect. Now we'll separate Task 3 and Task 4 carefully.

Step 2 — Temporarily save your completed version

Run this command in the VS Code terminal:

```
Copy-Item "$HOME\game_engine_final_backup.py" "$HOME\game_engine_task4.py"
```

This gives us a second copy of your complete code, so we can restore it after making the Task 3 commit.

Step 3 — Open the GitHub version of the game engine

Run:

```
git show 96900f7:game/game_engine.py
```

This displays the version from your Task 2 commit, before the history and maximum-attempt features were added.

Don't copy or change anything yet. Tell me when that command finishes. We'll use the correct version to create a genuine Task 3 commit, then add Task 4 separately.

User

i ran the command

ChatGPT

Good, Avi. Now we need to create a separate commit for Task 3 without losing the completed Task 4 code.

Step 3 — Open game/game_engine.py

1. In VS Code, click game/game_engine.py.
2. Select all the code with Ctrl + A.
3. **Don't delete anything yet.** We need to make sure Task 3 and Task 4 are separated correctly.

Your terminal command displayed the Task 2 version, which doesn't contain either new feature. We'll use it as the starting point for rebuilding the separate commits.

Please send me the output you got from `git show`

96900f7:game/game_engine.py` (a screenshot is fine). I'll give you the Task 3-only version to paste, then we'll commit it before restoring Task 4.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file  
000000007a1482108c5b90caf54dffa8?sharedconversation  
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size  
bytes":84302,"width":1464,"height":808,"metadata":{"sanitized":true}}
```

ChatGPT

You're in the right place, Avi. The screenshot shows Git is displaying the old version of game_engine.py in a terminal viewer. The : at the bottom means you're inside the Git pager.

Step 1 — Exit this screen

Press `q` on your keyboard.

This returns you to the PowerShell prompt.

Step 2 — Create the separate Task 3 commit

Let's avoid manually rebuilding the file from scratch. You already have your complete working code backed up.

First, run:

```
Copy-Item "$HOME\game_engine_final_backup.py" game/game_engine.py
```

Then we need to split the changes so the first commit contains only the guess-history feature and the second adds the maximum-attempt feature. Since your backup contains both, **don't commit it yet**.

Step 3 — Let me check the backup before we proceed

Run:

```
Select-String -Path "$HOME\game_engine_final_backup.py" -Pattern "max_attempts|game_over|guess
```

Send me the output. I'll use it to guide you through creating the two commits without losing your working implementation.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file  
00000000ee4c8210b20f7e8d275c07c5?sharedconversation  
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size  
bytes":544452,"width":2048,"height":722,"metadata":{"sanitized":true}}
```

ChatGPT

Perfect, Avi. The backup is safe, and we've confirmed it contains both `guess_history` and `max_attempts`.

We'll now create a genuine Task 3 commit first, then a separate Task 4 commit.

Step 4 — Create the Task 3-only version

Your terminal is ready. Copy and run this command to restore the version from your existing Task 2 commit:

```
git show 96900f7:game/game_engine.py | Set-Content game/game_engine.py
```

Now we need to add only the guess history feature. To avoid accidentally breaking the code, we'll do this carefully.

First, open `game/game_engine.py` in VS Code and send me a screenshot of the file. I will provide the exact Task 3-only code to paste. We'll commit it as:

```
Add recent guess history
```

Then we'll restore your backup, make the Task 4 commit separately, and push both commits to GitHub.

User

```
import random  
import pygame  
from game.text_box import TextBox  
  
class GameEngine:  
    def init(self, width, height):  
        self.width = width  
        self.height = height
```

```

self.secret_number = random.randint(1, 100)
self.attempts = 0

# Task 2: Track the possible range
self.min_range = 1
self.max_range = 100

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False

self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

self.font_title = pygame.font.SysFont(None, 42)
self.font_medium = pygame.font.SysFont(None, 28)
self.font_btn = pygame.font.SysFont(None, 26)

def submit_guess(self):
    if self.game_won:
        return

    # Task 1: Handle empty input safely
    if not self.input_box.text.strip():
        self.feedback_msg = "Please enter a number!"
        self.feedback_color = (255, 220, 80)
        return

    guess = int(self.input_box.text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        # Task 2: Narrow the minimum possible value
        self.minrange = max(self.minrange, guess + 1)
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)

    elif guess > self.secret_number:
        # Task 2: Narrow the maximum possible value
        self.maxrange = min(self.maxrange, guess - 1)
        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)

    else:
        self.feedback_msg = (
            f"CORRECT! Found in {self.attempts} attempts."
        )
        self.feedback_color = (80, 220, 90)
        self.game_won = True

```

```

def reset(self):
self.secret_number = random.randint(1, 100)
self.attempts = 0

# Reset the search range for a new game
self.min_range = 1
self.max_range = 100

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False
self.input_box.clear()

def handle_event(self, event):
self.inputbox.handleevent(event)

if event.type == pygame.KEYDOWN:
if event.key == pygame.K_RETURN:
self.submit_guess()
elif event.key == pygame.Kr and self.gamewon:
self.reset()

elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
if self.submit_btn.collidepoint(event.pos):
self.submit_guess()

def update(self):
pass

def render(self, screen):
screen.fill((30, 34, 42))

titlesurf = self.fonttitle.render(
"Number Guessing Arena", True, (245, 245, 245)
)
screen.blit(
title_surf,
(self.width // 2 - titlesurf.getwidth() // 2, 35),
)

attemptssurf = self.fontmedium.render(
f"Attempts: {self.attempts}", True, (180, 185, 195)
)
screen.blit(
attempts_surf,
(self.width // 2 - attemptssurf.getwidth() // 2, 95),
)

# Task 2: Display the current possible search range
rangesurf = self.fontmedium.render(

```

```

f"Possible range: {self.minrange} - {self.maxrange}",
True,
(180, 185, 195),
)
screen.blit(
range_surf,
(self.width // 2 - rangesurf.getwidth() // 2, 120),
)
self.input_box.render(screen)
pygame.draw.rect(
screen, (50, 150, 80), self.submitbtn, borderradius=6
)
pygame.draw.rect(
screen, (220, 220, 220), self.submit_btn,
width=2, border_radius=6
)
btntext = self.fontbtn.render(
"SUBMIT", True, (255, 255, 255)
)
screen.blit(
btn_text,
(
self.submitbtn.centerx - btntext.get_width() // 2,
self.submitbtn.centery - btntext.get_height() // 2,
),
)
feedbacksurf = self.fontmedium.render(
self.feedbackmsg, True, self.feedbackcolor
)
screen.blit(
feedback_surf,
(
self.width // 2 - feedbacksurf.getwidth() // 2,
235,
),
)
if self.game_won:
restartsurf = self.fontmedium.render(
"Press [R] to Start a New Game",
True,
(255, 220, 80),
)

```

```

screen.blit(
restart_surf,
(
self.width // 2 - restartsurf.getwidth() // 2,
295,
),
)

```

ChatGPT

Perfect, Avi. This is your Task 2 version. We'll add **Task 3 (Recent Guess History)** to this version first, commit it, and then add Task 4 separately.

Step 1 — Replace game/game_engine.py with this Task 3 version

This keeps Tasks 1 and 2 and adds the history tracker. **It does not include the maximum-attempt limit yet.**

```

<CodeBlock language="python" editable>
import random
import pygame
from game.text_box import TextBox

class GameEngine:
def init(self, width, height):
self.width = width
self.height = height
self.secret_number = random.randint(1, 100)
self.attempts = 0

# Task 2: Dynamic search range
self.min_range = 1
self.max_range = 100

# Task 3: Recent guess history
self.guess_history = []
self.max_history = 5

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False

self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

self.font_title = pygame.font.SysFont(None, 42)
self.font_medium = pygame.font.SysFont(None, 28)
self.font_btn = pygame.font.SysFont(None, 26)
self.font_small = pygame.font.SysFont(None, 24)

def submit_guess(self):

```



```

if self.game_won:
    return

# Task 1: Safely handle empty input
text = self.input_box.text.strip()
if not text:
    self.feedback_msg = "Please enter a number!"
    self.feedback_color = (255, 220, 80)
    return

try:
    guess = int(text)
except ValueError:
    self.feedback_msg = "Please enter a valid number!"
    self.feedback_color = (255, 220, 80)
    return

if not 1 <= guess <= 100:
    self.feedback_msg = "Enter a number between 1 and 100!"
    self.feedback_color = (255, 220, 80)
    return

self.attempts += 1
self.input_box.clear()

if guess < self.secret_number:
    self.minrange = max(self.minrange, guess + 1)
    result = "TOO LOW"
    self.feedback_msg = f"TOO LOW! (Guess was {guess})"
    self.feedback_color = (80, 160, 240)

elif guess > self.secret_number:
    self.maxrange = min(self.maxrange, guess - 1)
    result = "TOO HIGH"
    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)

else:
    result = "CORRECT"
    self.feedback_msg = (
        f"CORRECT! Found in {self.attempts} attempts."
    )
    self.feedback_color = (80, 220, 90)
    self.game_won = True

# Record each valid guess and its result
self.guess_history.append((guess, result))
self.guesshistory = self.guesshistory[-self.max_history:]

def reset(self):

```

```

self.secret_number = random.randint(1, 100)
self.attempts = 0
self.min_range = 1
self.max_range = 100
self.guess_history.clear()

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False
self.input_box.clear()

def handle_event(self, event):
    self.inputbox.handleevent(event)

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RETURN:
            self.submit_guess()
        elif event.key == pygame.Kr and self.gamewon:
            self.reset()

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_guess()

    def update(self):
        pass

    def render(self, screen):
        screen.fill((30, 34, 42))

        titlesurf = self.fonttitle.render(
            "Number Guessing Arena", True, (245, 245, 245)
        )
        screen.blit(
            title_surf,
            (self.width // 2 - titlesurf.getwidth() // 2, 35),
        )

        attemptssurf = self.fontmedium.render(
            f"Attempts: {self.attempts}", True, (180, 185, 195)
        )
        screen.blit(
            attempts_surf,
            (self.width // 2 - attemptssurf.getwidth() // 2, 95),
        )

        rangesurf = self.fontmedium.render(
            f"Possible range: {self.minrange} - {self.maxrange}",
            True,
            (180, 185, 195),

```

```

)
screen.blit(
range_surf,
(self.width // 2 - rangesurf.getwidth() // 2, 120),
)
self.input_box.render(screen)

pygame.draw.rect(
screen, (50, 150, 80), self.submitbtn, borderradius=6
)
pygame.draw.rect(
screen, (220, 220, 220), self.submit_btn,
width=2, border_radius=6
)
)
btntext = self.fontbtn.render(
"SUBMIT", True, (255, 255, 255)
)
screen.blit(
btn_text,
(
self.submitbtn.centerx - btntext.get_width() // 2,
self.submitbtn.centery - btntext.get_height() // 2,
),
)
)
feedbacksurf = self.fontmedium.render(
self.feedbackmsg, True, self.feedbackcolor
)
screen.blit(
feedback_surf,
(
self.width // 2 - feedbacksurf.getwidth() // 2,
235,
),
)
)
# Task 3: Display recent guesses with directional colors
historytitle = self.fontmedium.render(
"Recent Guess History", True, (245, 245, 245)
)
screen.blit(
history_title,
(
self.width // 2 - historytitle.getwidth() // 2,
290,

```

```

),
)
history_y = 330
for guess, result in reversed(self.guess_history):
    if result == "TOO LOW":
        color = (80, 160, 240)
    elif result == "TOO HIGH":
        color = (240, 100, 80)
    else:
        color = (80, 220, 90)
    historysurf = self.fontsmall.render(
        f"{guess} - {result}", True, color
    )
    screen.blit(
        history_surf,
        (
            self.width // 2 - historysurf.getwidth() // 2,
            history_y,
        ),
    )
    history_y += 27
if self.game_won:
    restartsurf = self.fontmedium.render(
        "Press [R] to Start a New Game",
        True,
        (255, 220, 80),
    )
    screen.blit(
        restart_surf,
        (
            self.width // 2 - restartsurf.getwidth() // 2,
            self.height - 40,
        ),
    )
)
</CodeBlock>

```

Step 2 — Commit Task 3 separately

Save the file using `Ctrl + S`, then run:

```

git add game/game_engine.py
git commit -m "Add recent guess history"

```

Don't push yet. We need to add Task 4 as its own commit first.

Step 3 — Confirm the commit

Run:

```
git log -3 --oneline
```

You should now see Add recent guess history above Implement dynamic search range.

Send me the output, and I'll guide you through restoring the full version and committing

Task 4: maximum attempts and Game Over separately.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file  
00000000f2208210a6705db295765ed7?sharedconversation  
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size  
bytes":589109,"width":2048,"height":960,"metadata":{"sanitized":true}}
```

ChatGPT

Perfect, Avi! ☐ Your screenshot confirms that **Task 3 has its own commit now:**

76eb13c — Add recent guess history

Your Task 1 and Task 2 commits are still intact. Now we'll create the separate Task 4 commit.

Step 1 — Restore your completed code

Run this command in your VS Code terminal:

```
Copy-Item "$HOME\game_engine_final_backup.py" game/game_engine.py
```

This restores the full version containing both the history tracker and maximum-attempt feature.

Step 2 — Check the restored code

Run:

```
Select-String -Path game/game_engine.py -Pattern "max_attempts|game_over"
```

You should see lines containing max_attempts and game_over.

Step 3 — Commit Task 4 separately

Run:

```
git add game/game_engine.py  
git commit -m "Add maximum attempts and game over state"
```

Now your local history should contain four separate task commits:

1. Fix empty input crash
2. Implement dynamic search range
3. Add recent guess history
4. Add maximum attempts and game over state

Step 4 — Push the corrected history

Because the combined commit was already pushed to GitHub earlier, we'll need to update

the remote history safely.

First, run:

```
git status
git log -5 --oneline
```

Send me that output before pushing. I'll verify the commit sequence and give you the correct push command so we don't accidentally overwrite anything.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file
00000000aabb48210bc7c7d3338ad801f?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":540622,"width":2048,"height":929,"metadata":{"sanitized":true}}
```

ChatGPT

Perfect, Avi. ☐ The backup has been restored successfully, and the output confirms that game/game_engine.py contains the Task 4 code (max_attempts and game_over).

Now we need to commit Task 4 separately.

Step 1 — Commit Task 4

Run these commands in your VS Code terminal:

```
git add game/game_engine.py
git commit -m "Add maximum attempts and game over state"
```

Step 2 — Verify your commits

Run:

```
git log -5 --oneline
git status
```

You should see these four task commits in order:

1. Add maximum attempts and game over state
2. Add recent guess history
3. Implement dynamic search range
4. Fix empty input crash

Your working tree should also be clean.

Step 3 — Stop before pushing

Your GitHub repository still contains the old combined commit. **Don't run `git push` yet.** Once you send me the output from Step 2, I'll give you the safest command to update GitHub with the corrected commit history.

User

```
{"contenttype":"imageassetpointer","assetpointer":"sediment://file
000000009fd08210b7b78fbc75ace919?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":108071,"width":1550,"height":506,"metadata":{"sanitized":true}}
```

ChatGPT

Excellent, Avi! ☐ Your commit history is now correctly separated.

Your screenshot confirms these four commits:

Task	Commit
Task 1 — Empty input crash	Fix empty input crash
Task 2 — Dynamic search range	Implement dynamic search range
Task 3 — Guess history	Add recent guess history
Task 4 — Maximum attempts	Add maximum attempts and game over state

Step 1 — Update GitHub

Because we rewrote the local history, GitHub still has the old combined commit. First, run:

```
git status
```

If the working tree is clean, run:

```
git fetch origin
git log --oneline origin/main -3
```

This lets us verify the remote history before updating it.

Step 2 — Push the corrected history

If GitHub still shows the old combined commit, run:

```
git push --force-with-lease origin main
```

This safely updates the remote branch only if it hasn't changed unexpectedly. Don't use plain `--force`.

Step 3 — Verify

Refresh your [GitHub repository](https://github.com/Avi2106/60numberguess) and check that all four task commits appear separately.

Then you only need to finish the required before/after videos and share your ChatGPT conversation link.

Do Step 1 first and send me the output. I'll verify whether the remote is ready to update.

User

```
{"contentType":"imageassetpointer","assetpointer":"sediment://file
00000000c78c8210aaeed042c6906094?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":155114,"width":1602,"height":838,"metadata":{"sanitized":true}}
```

ChatGPT

Avi, this is exactly what we expected. Your local branch has the two separate commits, while GitHub still has the old combined commit. **We need to replace the remote history with your corrected history.**

Your screenshot confirms:

- Local history has Add recent guess history and Add maximum attempts and game over state separately.
- GitHub still has Add guess history and maximum attempts as one commit.
- Your .gitignore file is untracked, but that's not blocking the history fix.

Step 1 — Push the corrected commits

Run this command in your VS Code terminal:

```
git push --force-with-lease origin main
```

This updates GitHub with your local commit history, provided the remote hasn't changed since your fetch. It's safer than plain --force.

Step 2 — Verify the result

If the push succeeds, run:

```
git fetch origin
git log --oneline origin/main -5
```

You should see these four task commits separately:

1. Add maximum attempts and game over state
2. Add recent guess history
3. Implement dynamic search range
4. Fix empty input crash

Step 3 — Clean up .gitignore

After the history is fixed, you can include your cache exclusion:

```
git add .gitignore
git commit -m "Ignore Python cache files"
git push origin main
```

Do Step 1 first and tell me whether the push succeeds or shows an error.

User

```
{"contentType":"imageassetpointer","assetpointer":"sediment://file
0000000034948210ad480d9933a00c8d?sharedconversation
id=6ac87bb0-6fa8-83ee-8848-024039140b15","size
bytes":246332,"width":1880,"height":962,"metadata":{"sanitized":true}}
```

ChatGPT

Perfect, Avi! ☐ **The separate commits are now successfully pushed to GitHub.**

Your screenshot confirms that origin/main and your local main point to the same commit, and all four task commits are present.

Final Git verification

| Task | Commit | Status |

|---|---|---|

| Task 1 — Fix empty input crash | 59eadd8 | <badge color="success">Pushed</badge> |

| Task 2 — Dynamic search range | 96900f7 | <badge color="success">Pushed</badge> |
| Task 3 — Recent guess history | 76eb13c | <badge color="success">Pushed</badge> |
| Task 4 — Maximum attempts and Game Over | 562ce48 | <badge
color="success">Pushed</badge> |

Your repository: [Avi2106/60numberguess](https://github.com/Avi2106/60numberguess)

What's left?

1. **Before video:** Record 10 seconds showing the original empty-input crash. If you didn't record it before fixing the bug, you may need to restore the original version temporarily to demonstrate it.
2. **After video:** Record 10 seconds showing the fixed input, narrowing range, guess history, and Game Over feature.
3. **Chat history link:** Include the link to this conversation with the full interaction history.
4. **Optional cleanup:** Your .gitignore is still untracked. It doesn't affect the four task commits, but you can add it separately if you want.

One last check: run `git status`. Your code should be clean apart from the untracked .gitignore.

Have you recorded the before video yet? That's the next important step for your submission.

User

now i have to record the after video right

ChatGPT

Yes, Avi! ☐ Your code is pushed to GitHub with all four tasks in separate commits. Now let's record the **10-second AFTER video** showing your completed game.

Step 1 — Start the game

In your VS Code terminal, run:

```
python main.py
```

Step 2 — Record the video

On Windows, press ``Win + Alt + R`` to start recording with Xbox Game Bar. Press it again to stop recording.

If that shortcut doesn't work, open Xbox Game Bar with `Win + G` and use the Capture widget.

Step 3 — Show these features in the recording

```
<box border radius="lg" padding={3} gap={3}>
<row align="start" gap={3}>
<AsyncImage query="Pygame number guessing game dark interface input box submit
button" aspectRatio="4:3" maxWidth="104px"/>
<box flex="1" gap={1}>
```

1. Empty-input fix

```

<text color="secondary" size="sm">Click Submit with the input box empty. Show that a
warning appears and the game stays open.</text>
</box>
</row>
<divider color="subtle"/>
<row align="start" gap={3}>
<AsyncImage query="number guessing game interface showing too high too low hints and
possible range" aspectRatio="4:3" maxWidth="104px"/>
<box flex="1" gap={1}>

```

2. Dynamic range and history

```

<text color="secondary" size="sm">Make two or three guesses. Show the possible range
narrowing and the color-coded guess history updating.</text>
</box>
</row>
<divider color="subtle"/>
<row align="start" gap={3}>
<AsyncImage query="number guessing game dark interface game over screen reveal
secret number" aspectRatio="4:3" maxWidth="104px"/>
<box flex="1" gap={1}>

```

3. Game Over feature

```

<text color="secondary" size="sm">If you can fit it into the demonstration, show the
Game Over message after ten incorrect valid guesses, including the revealed
number.</text>
</box>
</row>
</box>

```

Recording tip: You only have 10 seconds, so prioritize showing the working game, narrowing range, history, and Game Over. You can demonstrate the empty-input warning first if you can fit everything into the clip.

Step 4 — Save and submit

Save the recording with a name like `after_changes.mp4`.

Your submission checklist:

- [] Before video showing the original crash
- [] After video showing the completed game
- [] ChatGPT conversation link
- [] Updated GitHub repository

Once you've recorded it, tell me. I'll help you make sure the submission is complete.

If you want, I can:

- Summarize this conversation
- Show me how to record the after video

- Visualize game feature progression